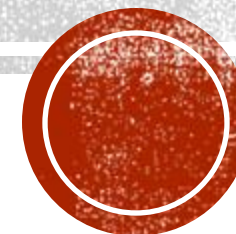
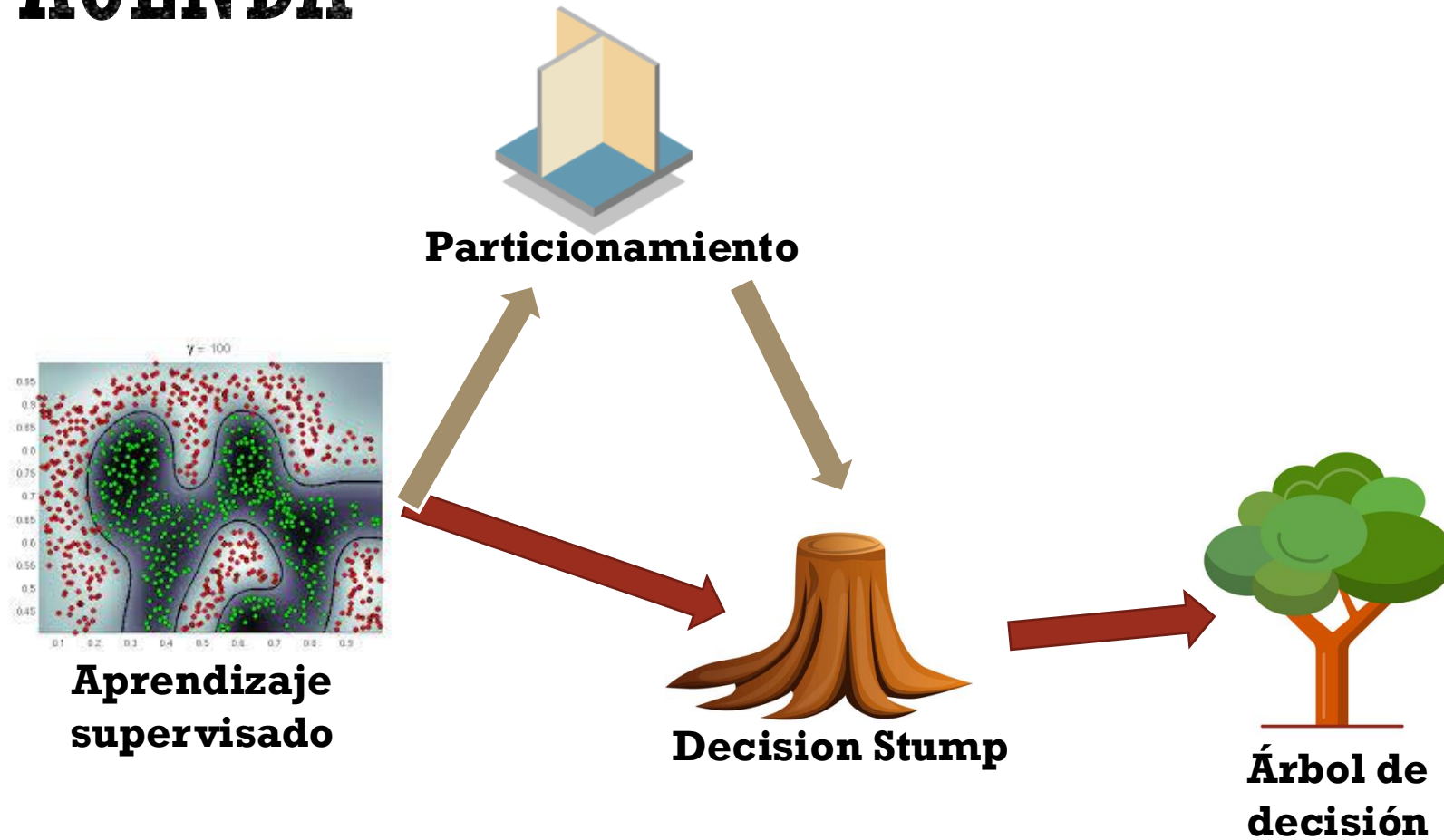


APRENDIZAJE SUPERVISADO





ÁRBOLES DE DECISIÓN: ALGORITMO

Dividir & conquistar: se divide de manera incremental el espacio muestral en regiones no superpuestas que son representadas por los nodos del árbol:

- **Seleccionar un factor** que particione **óptimamente** el nodo actual, minimizando una función de impureza
- **Dividir** el conjunto de datos del nodo con respecto a los valores del factor seleccionado y crear los nodos hijo correspondientes
- **Repetir recursivamente** hasta que
 - todas las instancias de los nodos hoja sean de la misma clase
 - no existan más atributos para particionar
 - se llegue a un criterio de parada definido (pre-poda)



ÁRBOLES DE DECISIÓN

- Los datos pueden ser **categoricos o numéricos**. Las variables continuas deben ser **discretizadas**.
- Un árbol de decisión se puede representar como un conjunto de **reglas** booleanas
- Una nueva instancia se clasifica **siguiendo las ramas** del árbol
- Ideal para casos en los cuales un pequeño número de atributos provee gran cantidad de información
- Prueba diferentes atributos categoricos para aprender una clase, hace una selección automática de variables importantes.
- No se basa en ninguna noción de distancia, el modelo es **indiferente** a nociones de **normalización**



ÁRBOLES DE DECISIÓN

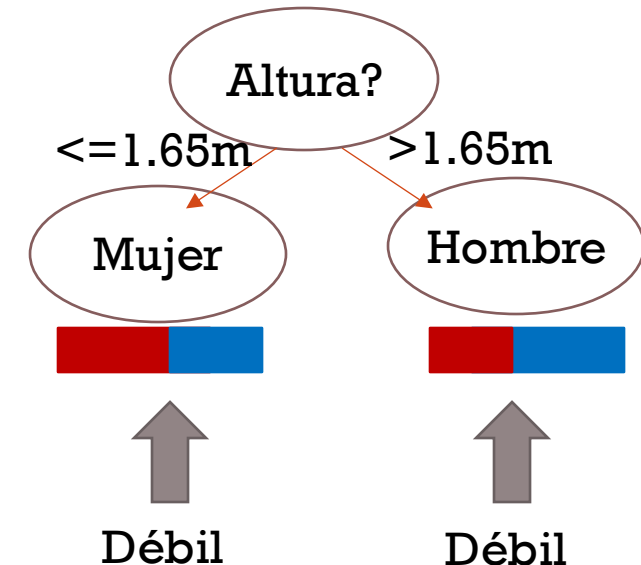
- Algoritmo **greedy**: busca óptimos locales a cada etapa, los cuales no necesariamente son óptimos globales
- **Simple** de implementar
- Puede ser usado para **clasificación y regresión**
- Clasificador **no lineal** (considera interacciones no lineales entre las variables predictivas)
- Mejor **rendimiento** en contextos no lineales (con datos estructurados)
- Modelo **escalable** (procesamiento distribuido)



DECISION STUMP

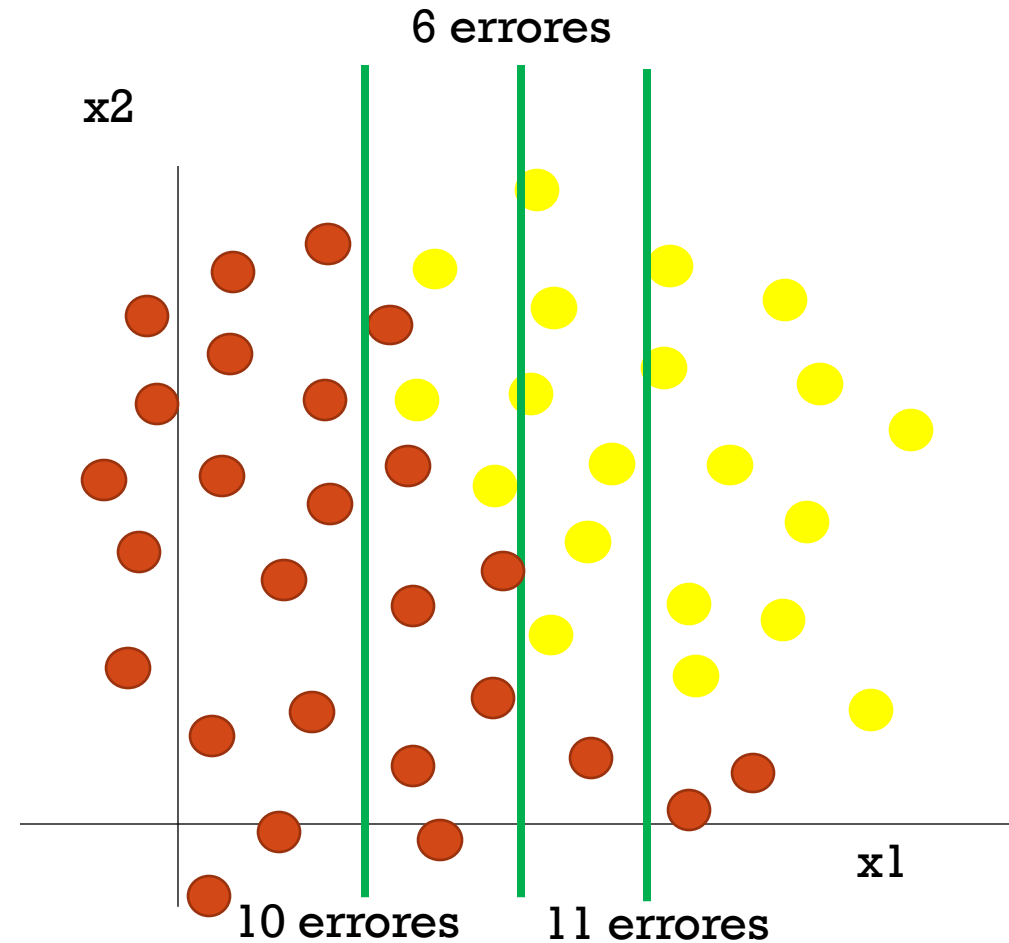


- Encuentra la mejor partición considerando una única variable predictiva
 - Árbol de decisión de un solo nivel
- Es un “**aprendiz débil**”, tiene una sola regla de decisión. Por ejemplo:
 - Las personas que miden más de 1,65 metros son hombres, y las que no, son mujeres
- Ampliamente utilizado en modelos de ensamble (especialmente Boosting)

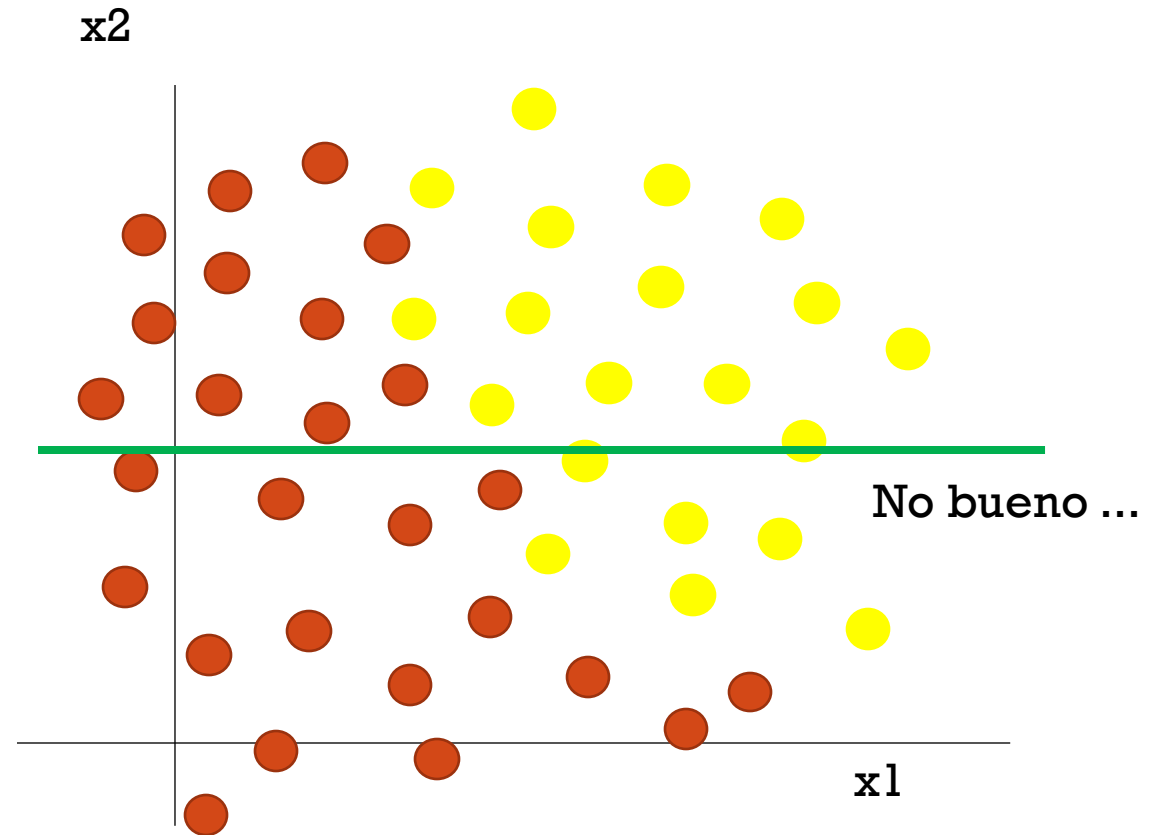


DECISION STUMP

- La partición en variables numéricas solo se puede realizar perpendicular a los ejes.
- Se busca minimizar un error de clasificación/regresión.
- Se busca todas las particiones posibles en las variables predictoras.
- ¿Cómo serían las reglas y el número de errores de los siguientes clasificadores?



- La partición en variables numéricas solo se puede realizar perpendicular a los ejes.
- Se busca minimizar un error de clasificación/regresión.
- Se busca todas las particiones posibles en las variables predictoras.
- ¿Cómo serían las reglas y el número de errores de los siguientes clasificadores?



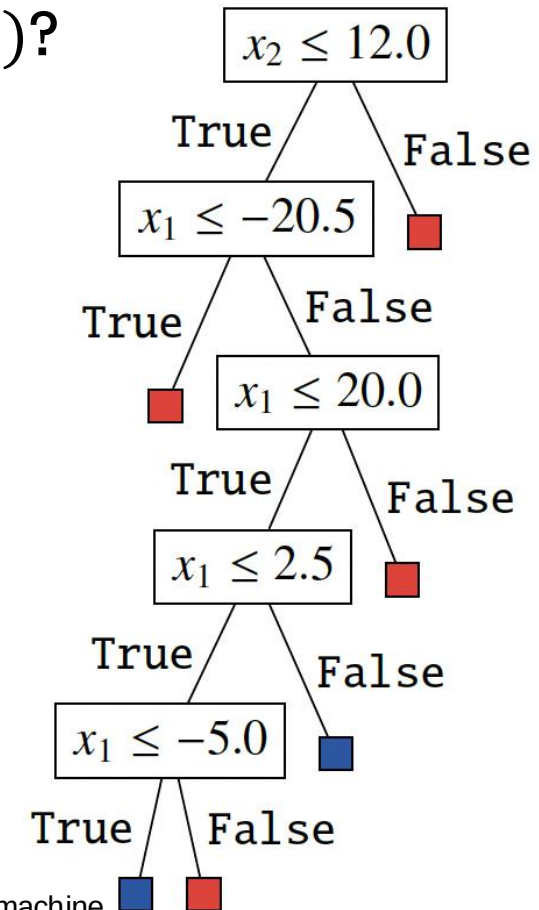
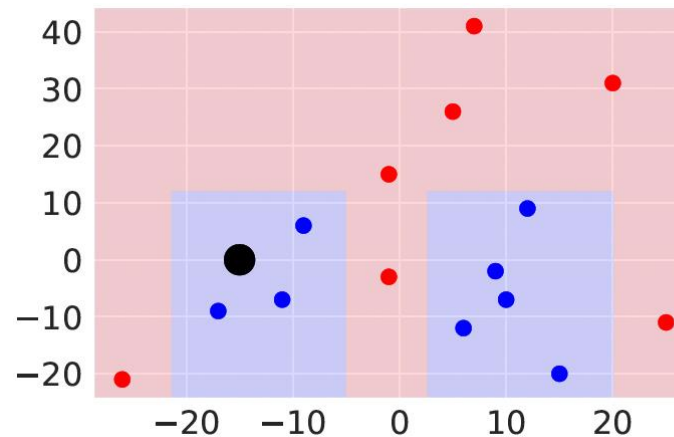
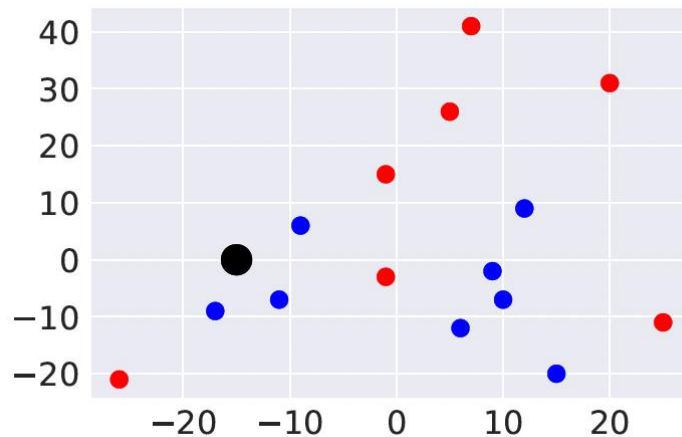
TALLER: DECISIÓN STUMP CHURN

Trabajaremos brevemente en el cuaderno Dec_Stump-Ejemplo



ARBOLES DE DECISION: EJEMPLO

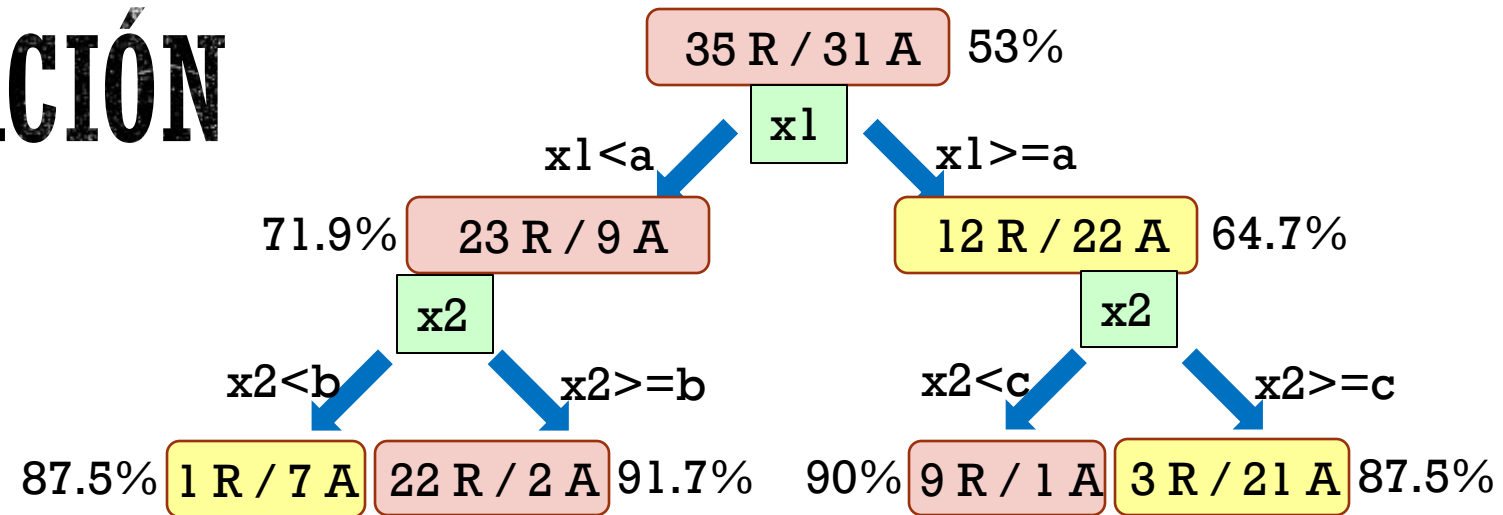
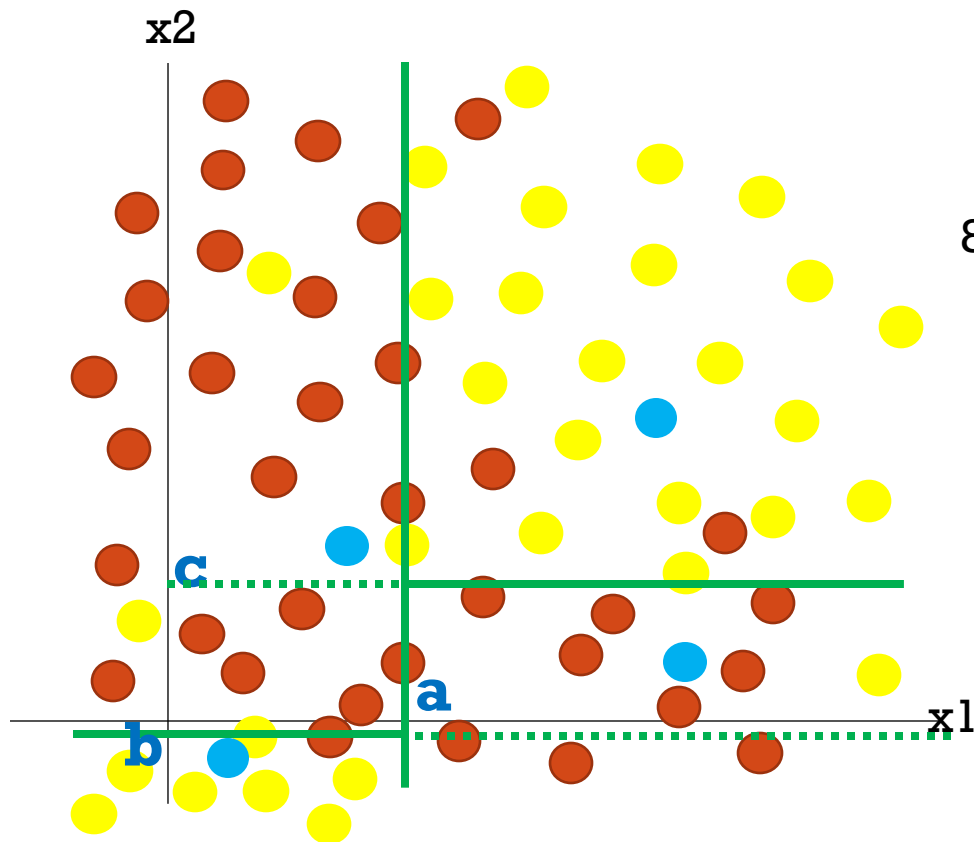
¿Cómo clasifica el punto negro: $x = (x_1, x_2) = (15, 0)$?



Kroese, D. P., Botev, Z., & Taimre, T. (2019). Data science and machine learning: mathematical and statistical methods.



ÁRBOLES DE DECISIÓN: CLASIFICACIÓN



Paso	Accuracy
Raíz	35/66 = 53%
1era partición	45/66 = 68.2%
2a partición (rama izq.)	51/66 = 77.3%
3a partición (rama der.)	59/66 = 89.4%

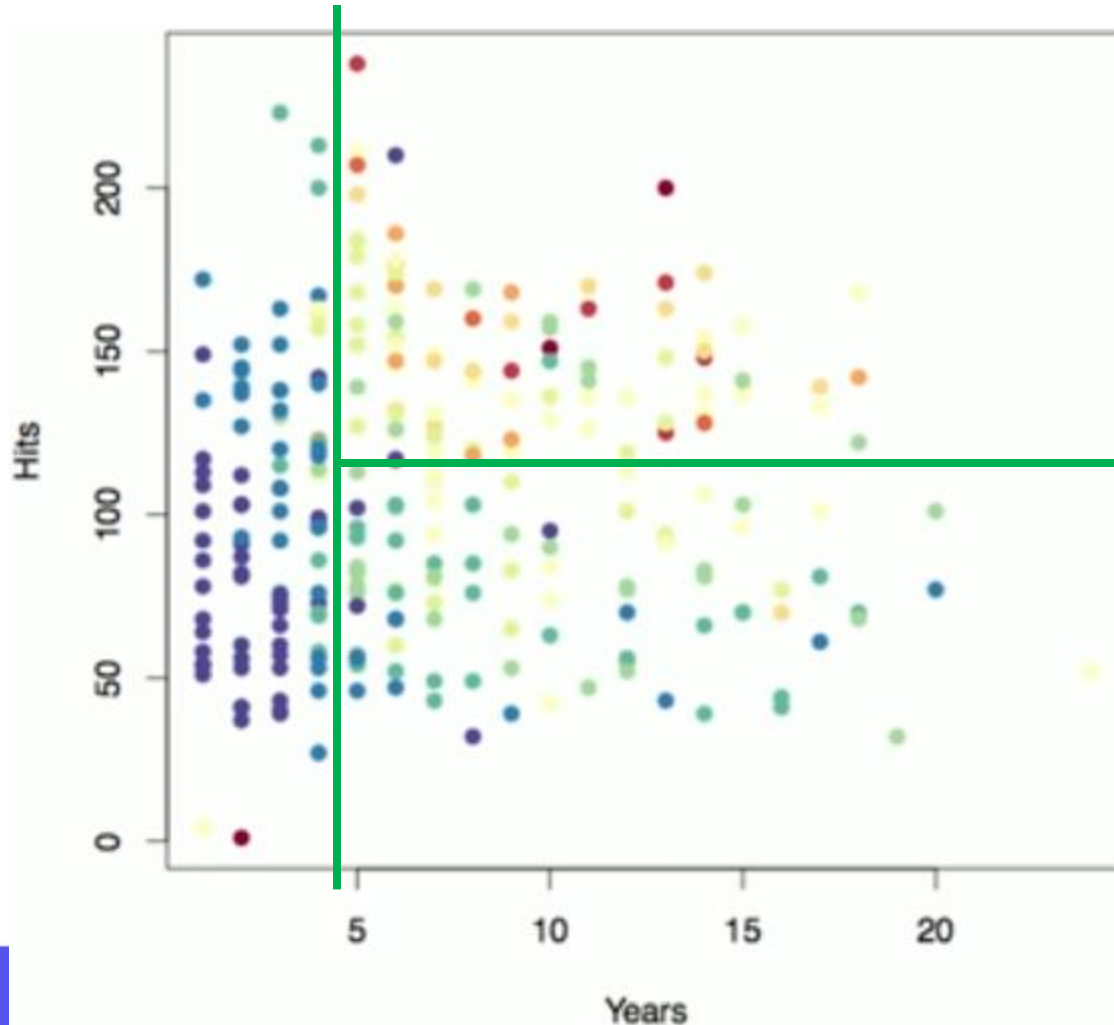
Se minimiza localmente una función de costo que considera la **impureza** de los nodos terminales del árbol



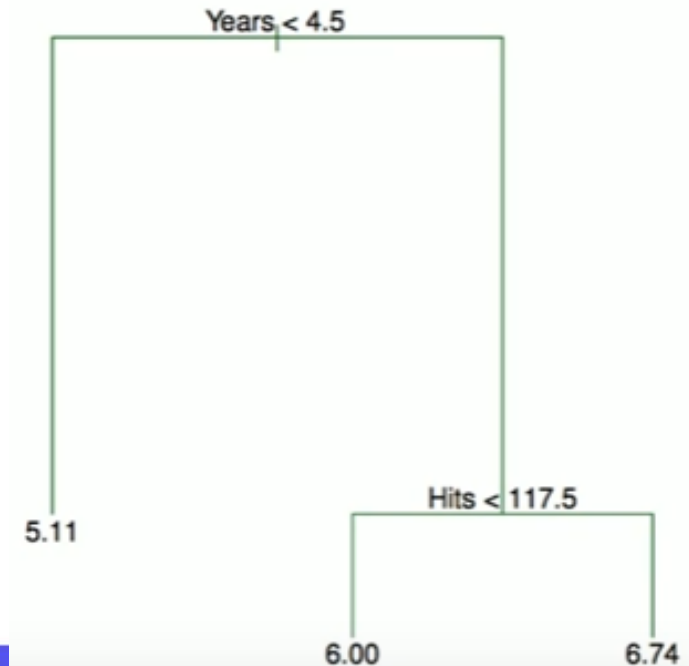
ÁRBOLES DE DECISIÓN: REGRESIÓN

Evolución de los salarios de beisbolistas (color) con respecto a años de experiencia (abscisa) y número de bateos exitosos (ordenada).

Se minimiza localmente $\sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2$



ISLR, 2013



CASO DE ESTUDIO: PROSPECCIÓN DE CLIENTES

Una compañía de seguros quiere contactar con clientes potenciales de una base de datos de **100.000** personas que acaban de adquirir para ofrecerles un plan. Tienen información de campañas anteriores incluyendo diferentes características como edad, sexo y salarios, así como una indicación de si la oferta tuvo éxito o no.

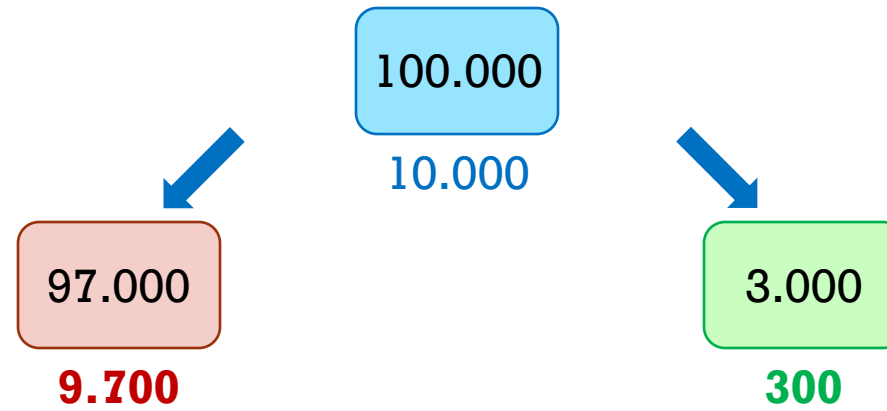
Teniendo en cuenta el costo de contactarlos, sólo se puede utilizar **10.000** clientes potenciales.

Sabemos por campañas anteriores que sólo el **3%** de las personas contactadas adquirieron el plan, pero esta tasa varía considerablemente si comenzamos a considerar subpoblaciones con características particulares como: edad, salario,



CASO DE ESTUDIO: PROSPECCIÓN DE CLIENTES

Escoja 10.000 clientes potenciales aleatoriamente

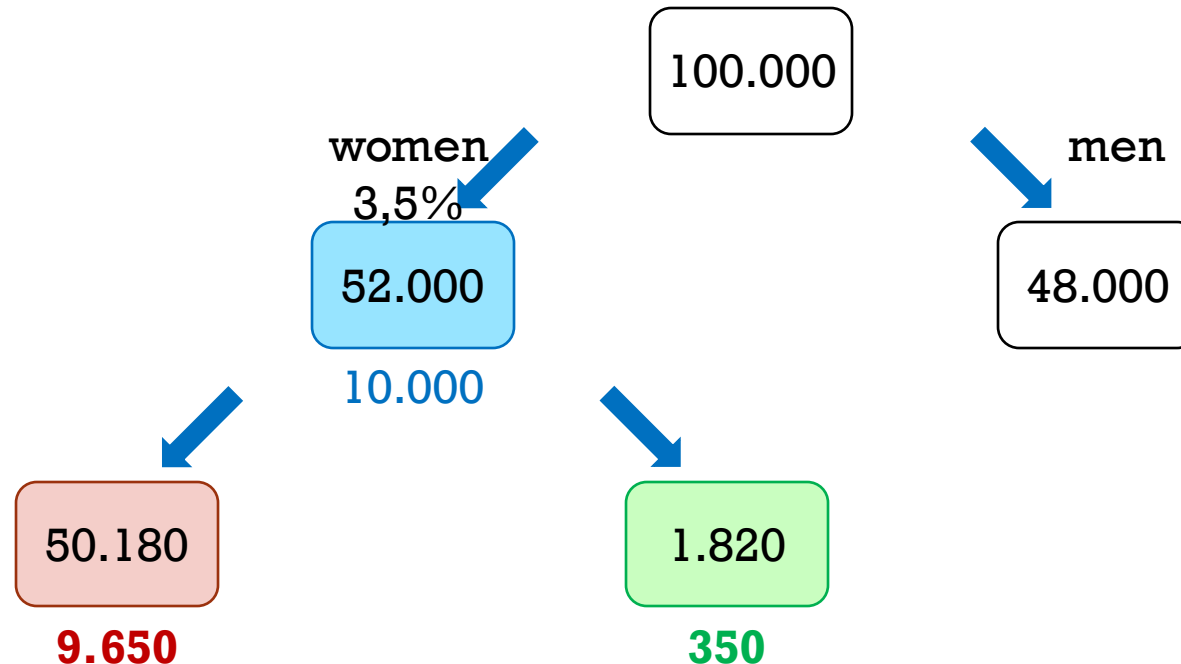


Éxito: 3% (300)



CASO DE ESTUDIO: PROSPECCIÓN DE CLIENTES

Las mujeres tienen más probabilidades de adquirir seguros (3,5%) y hay 52.000 mujeres en la BD



Éxito: 3,5% (350)



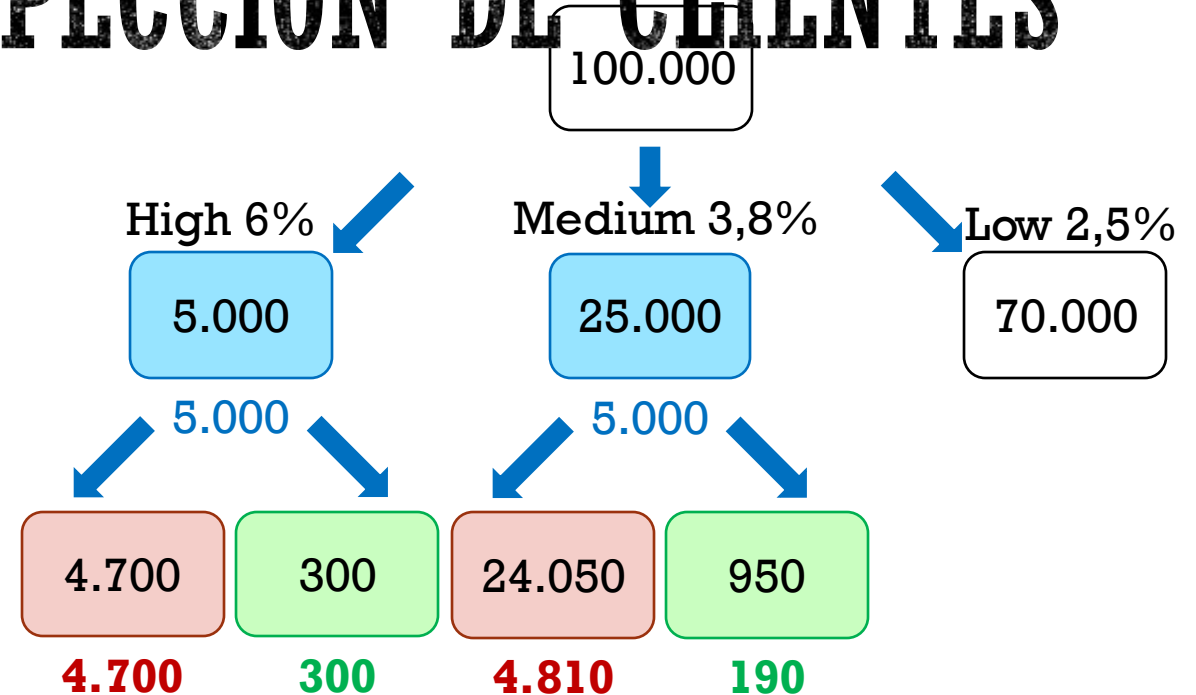
CASO DE ESTUDIO: PROSPECCIÓN DE CLIENTES

Las tasas de éxito por tipo de salario son:

Alto (5.000 en grupo): 6%

Medio (25.000 en grupo): 3,8%

Bajo (70.000 en grupo): 2,5%



Éxito: 4,9% (490)



CASO DE ESTUDIO: PROSPECCIÓN DE CLIENTES

Las tasas de éxito por tipo de salario son:

Alto (5.000 en grupo): 6%

Medio (25.000 en grupo): 3,8%

Bajo (70.000 en grupo): 2,5%

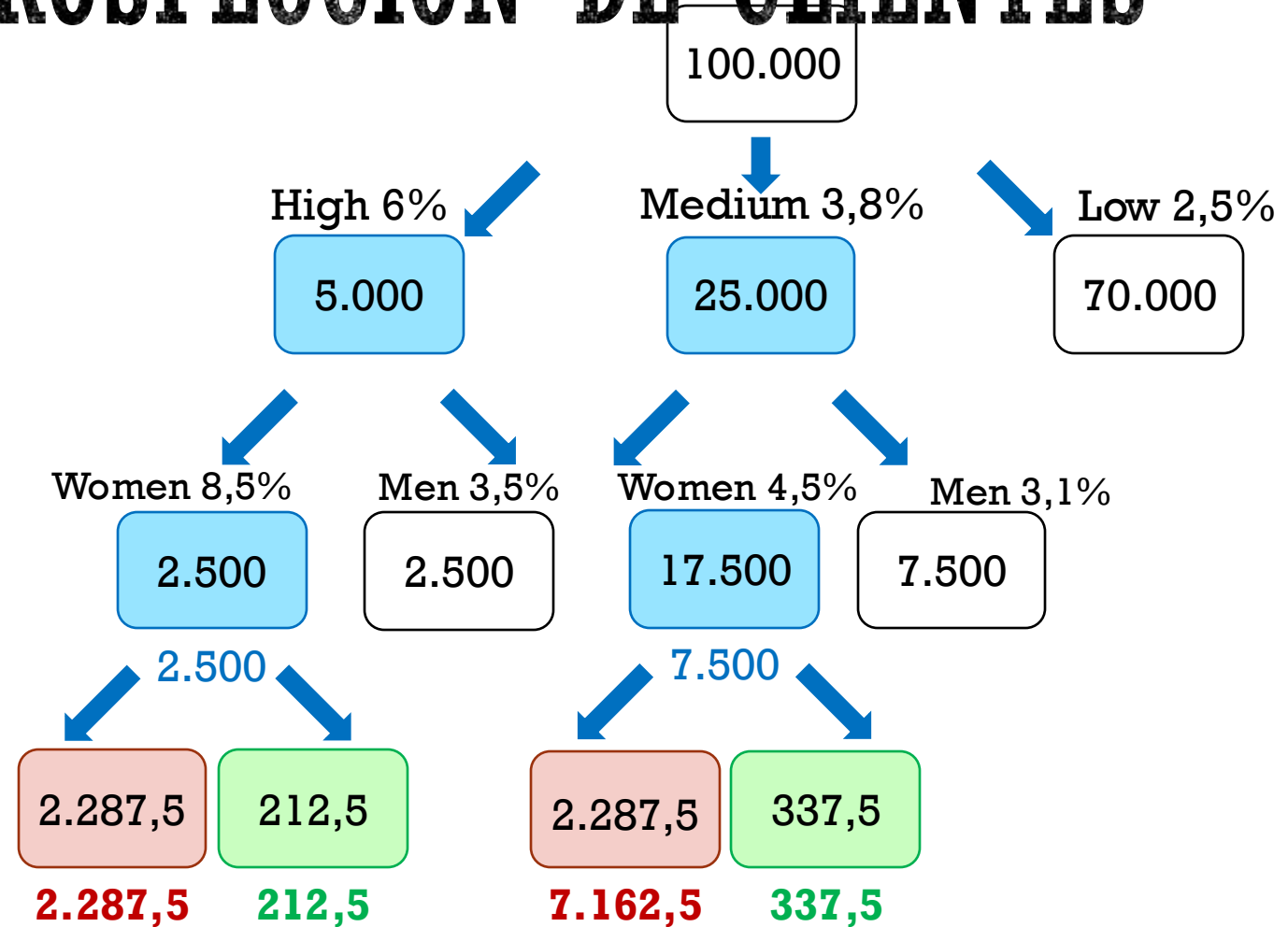
Mujeres con salario alto (2.500): 8,5%

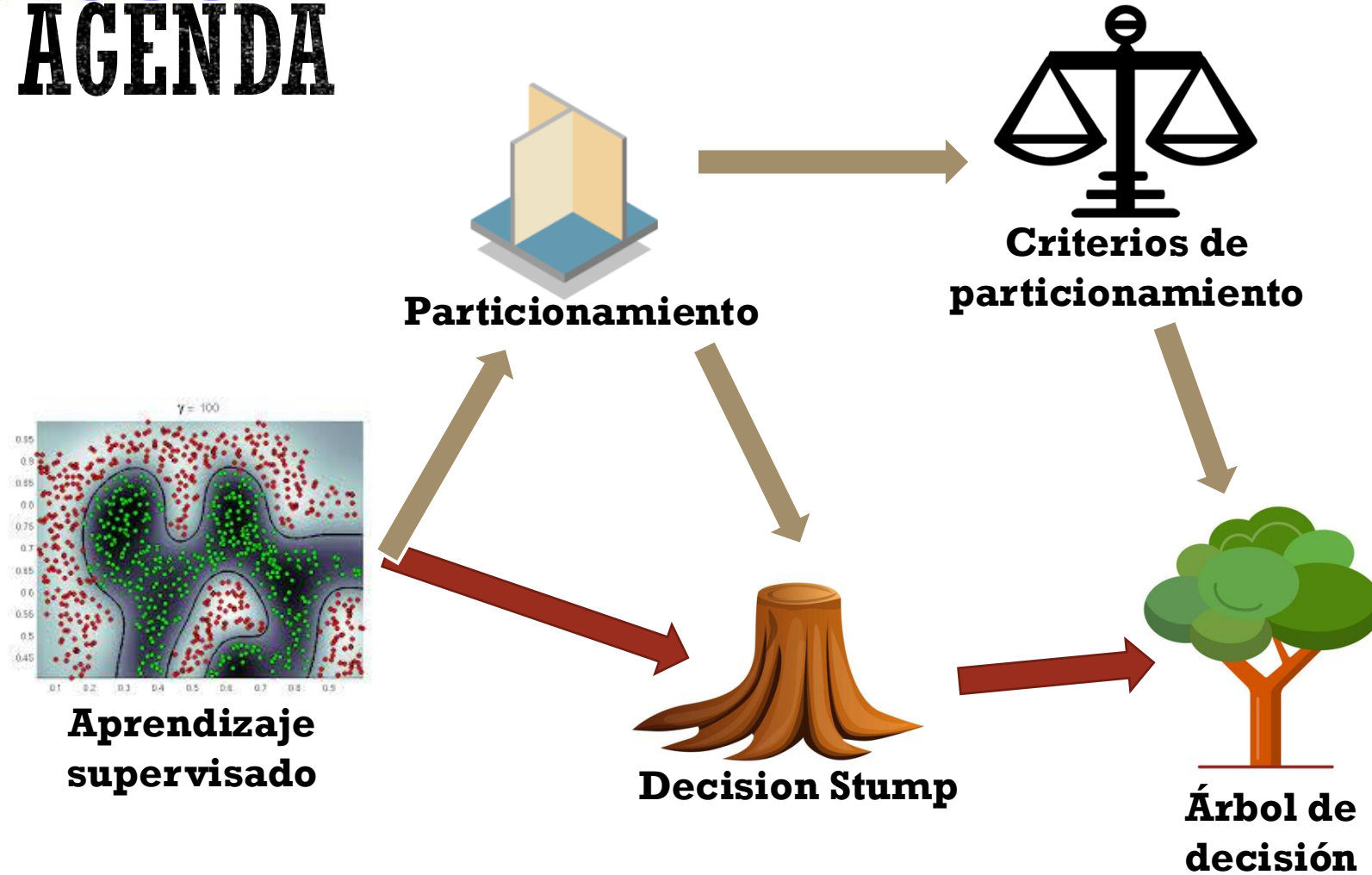
Hombres con salario alto (2.500): 3,5%

Mujeres con salario medio (17.500): 4,5%

Hombres con salario medio (7.500): 3,1%

¿Cómo hacemos esto de forma inteligente?



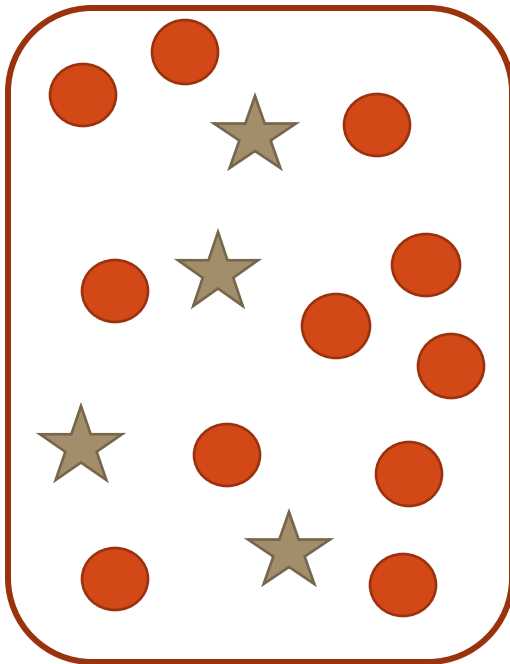


ÁRBOLES DE DECISIÓN (CRITERIOS)

- **Existen diferentes criterios para determinar el mejor atributo de particionamiento en cada nodo →**
 - **CART** (Classification And Regression Trees - Sklearn). Sólo particiones binarias, usando la métrica de impureza Gini para clasificación y la reducción de varianza para regresión
 - **ID3** (Iterative Dichotomizer). Encuentra el atributo que maximiza la ganancia de información (en cada nodo).
 - **C4.5 (C5.0)**. Extensión de ID3. Considera atributos continuos y discretos. Convierte los árboles ID3 en conjuntos de reglas if-then.

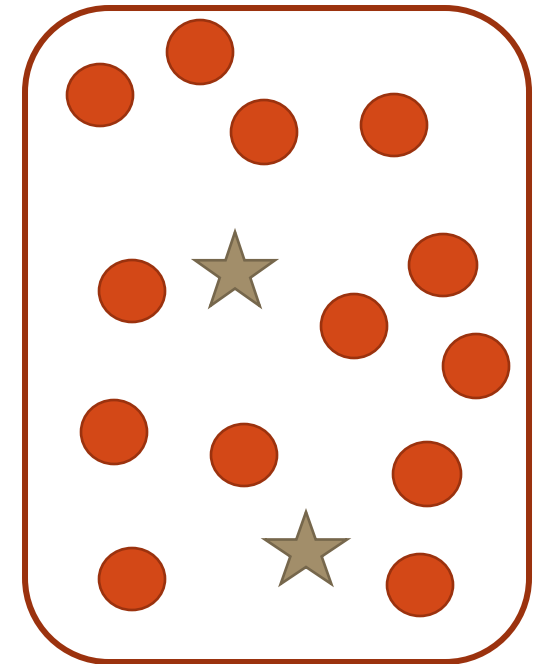


ÁRBOLES DE DECISIÓN: ID3



¿Cuál conjunto de datos presenta mayor desorden?

- Entropía, como medida de desorden
- Búsqueda de particiones cada vez mas puras
 - Reducción del desorden
 - Ganancia de información



ÁRBOLES DE DECISIÓN: ID3

Utiliza métricas de la **teoría de información**

- Seleccionar el atributo que más reduce el desorden en la variable objetivo del dataset

- Entropía:

$$H(Y) = -\sum_i p(Y = y_i) * \log_2(p(Y = y_i))$$

$H(Y) = 0$, si no hay errores de clasificación

- Ent.Cond.

$$H(Y|X = x_j)$$

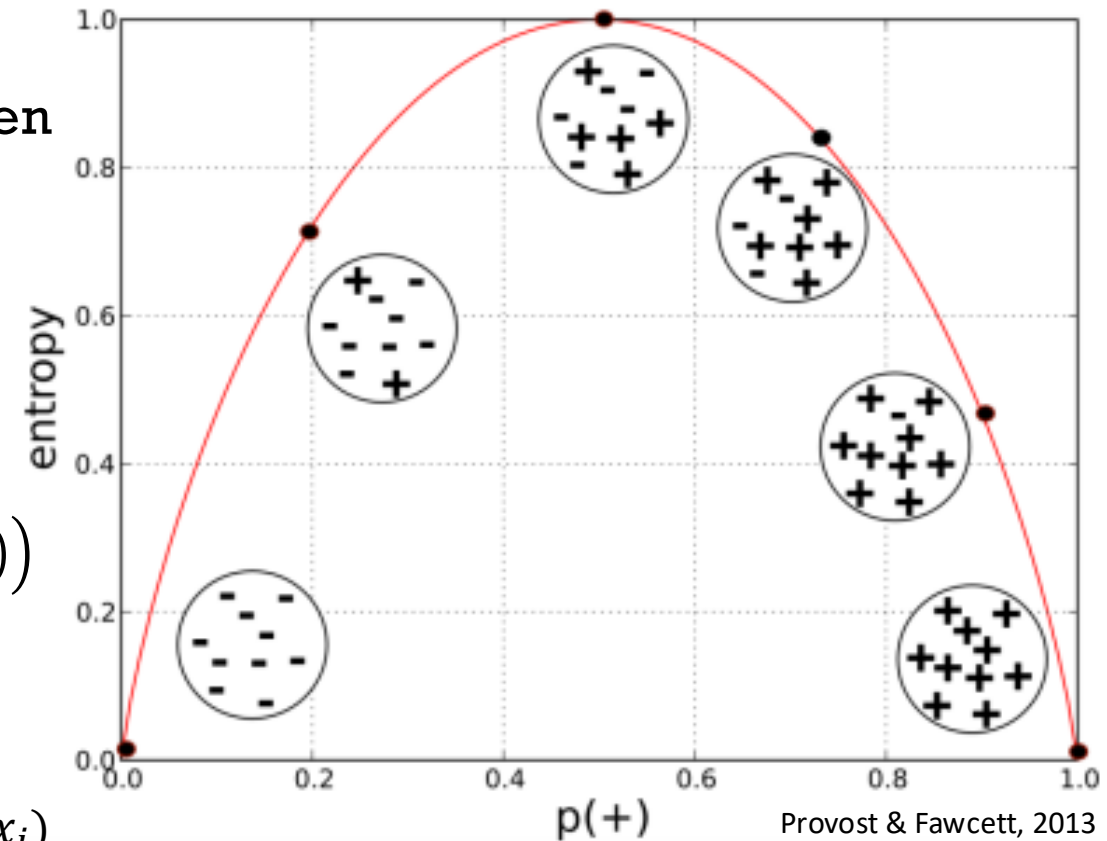
$$= -\sum_i p(Y = y_i|X = x_j) * \log_2(p(Y = y_i|X = x_j))$$

- Ent.Cond.Prom.

$$H(Y|X) = \sum_j p(X = x_j) * H(Y|X = x_j)$$

- Ganancia de información

$$Gain(Y, X = x_j) = H(Y) - \sum_j p(X = x_j) * H(Y|X = x_j)$$



ÁRBOLES DE DECISIÓN: ID3

- **Ventajas ID3**
 - Fácil de interpretar para árboles poco profundos. Soporta la presencia de valores faltantes y es robusto al ruido.
 - Puede manejar fácilmente atributos redundantes o irrelevantes (a menos que los atributos interactúen).
- **Desventajas ID3**
 - El espacio de posibles árboles de decisión crece exponencialmente.
 - No tiene en cuenta las interacciones entre atributos.
 - Cada frontera de decisión involucra a un solo atributo.



TALLER: ÁRBOLES DE DECISIÓN ID3

	outlook	temperature	humidity	windy	play
1	sunny	hot	high	FALSE	no
2	sunny	hot	high	TRUE	no
3	overcast	hot	high	FALSE	yes
4	rainy	mild	high	FALSE	yes
5	rainy	cool	normal	FALSE	yes
6	rainy	cool	normal	TRUE	no
7	overcast	cool	normal	TRUE	yes
8	sunny	mild	high	FALSE	no
9	sunny	cool	normal	FALSE	yes
10	rainy	mild	normal	FALSE	yes
11	sunny	mild	normal	TRUE	yes
12	overcast	mild	high	TRUE	yes
13	overcast	hot	normal	FALSE	yes
14	rainy	mild	high	TRUE	no

Dataset de clima: 14 instancias, 4 variables independientes para predecir una clase con 2 categorías posibles

¿Cuál es el mejor atributo para particionar el dataset?

1. Calcular la entropía de la clase (“play”)

Play (Y)				H
p(Y=no)	35.7%	-p(Y=no) log p(Y=no)	0.53	0.940
p(Y=yes)	64.3%	-p(Y=yes) log p(Y=yes)	0.41	

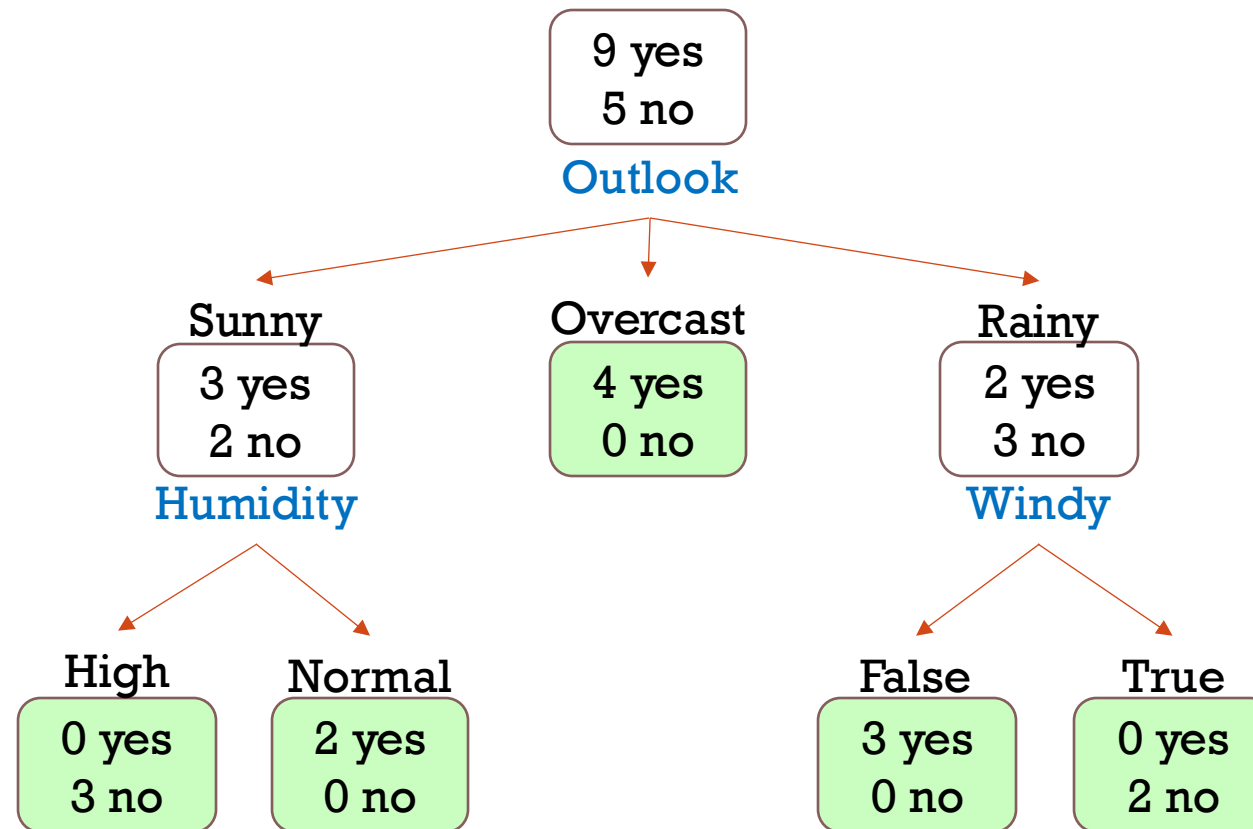
2. Calcular la entropía condicional para cada atributo y su ganancia de información

Outlook								GAIN
p(sunny)	35,7%	p(yes sunny)	40,0%	p(no sunny)	60,0%	0,971	0,694	0,247
p(overcast)	28,6%	p(yes overcast)	100,0%	p(no overcast)	0,0%	0,000		
p(rainy)	35,7%	p(yes rainy)	60,0%	p(no rainy)	40,0%	0,971		

2. Particionar según el atributo con mayor ganancia de información
3. Parar si todas las hojas son puras o ya no hay mas atributos



TALLER: ÁRBOLES DE DECISIÓN ID3



ÁRBOLES DE DECISIÓN: CART Posgrados

(CLASIFICATION AND REGRESSION TREE)

- **Árboles binarios**
- Manejo de datos faltantes: Utiliza varias variables de particionamiento **sustitutas** (surrogate) para suplantar la variable cuyo valor es faltante
- **Gini** como criterio de impureza para el particionamiento:
 - 0 pureza perfecto: todas las instancias de la misma clase
 - 0.5 impureza: distribución equitativa de las instancias entre ambas clases
- Algoritmo
 - Para cada atributo
 - Para cada posible split binario del atributo
 - Calcular el Gini para ambos subnodos
$$gini = 1 - \sum p_i^2$$
 donde p es la probabilidad de cada clase.
 - Calcular el promedio ponderado del Gini de las particiones, $avg. gini = \sum w_i * gini_i$ donde w_i es la proporción de instancias para cada clase.
 - Seleccionar el atributo con el menor promedio de Gini



ÁRBOLES DE DECISIÓN: CART

- **Ventajas:**
 - Son no paramétricos y no lineales.
 - Realizan implícitamente selección de características.
 - Los valores atípicos no tienen un efecto significativo en la predicción.
 - Requiere una supervisión mínima y produce modelos fáciles de entender.
- **Limitaciones:**
 - Sobreajuste.
 - Alta variación.
 - Bajo sesgo.
 - La estructura del árbol puede ser inestable.



ÁRBOLES DE DECISIÓN: FORMULACION MATEMÁTICA

- Dado un conjunto de datos $(x_i, y_i) \in R^{n \times n}$ un árbol de decisión particiona recursivamente el espacio de atributos.
 - Registros con la misma etiqueta quedan en la misma región
- Sea Q_m el conjunto de n_m registros en el nodo m .
 - Por cada partición $\Theta = (j, t_m)$ para un atributo j y umbral t_m :
 - $Q_m^{left}(\Theta) = \{(x, y): x_j \leq t_m\}$
 - $Q_m^{right}(\Theta) = Q_m - Q_m^{left}(\Theta)$
 - La calidad de la partición se mide optimizando una función de impureza o pérdida $H()$

$$\min_{\Theta} G(Q_m, \Theta) = \frac{n_m^{left}}{n_m} H(Q_m^{left}(\Theta)) + \frac{n_m^{right}}{n_m} H(Q_m^{right}(\Theta))$$

de forma recursiva hasta alcanzar un criterio de parada (max_Depth, min_samples, ...)



ÁRBOLES DE DECISIÓN: FORMULACION MATEMATICA

- CRITERIO DE CLASIFICACIÓN

- Si tenemos K etiquetas, sea

$$p_{mk} = \frac{1}{n_m} \sum_{y \in Q_m} I(y = k)$$

la proporción de la clase k en el nodo m . (Si m es un nodo terminal $p_{mk} = \text{predict_proba}$)

- Medidas de impureza:

- $H(Q_m) = \sum_k p_{mk}(1 - p_{mk}) \leftarrow \text{Gini}$
 - $H(Q_m) = -\sum_k p_{mk} \log(p_{mk}) \leftarrow \text{Log Loss o entropía}$

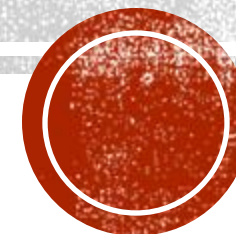
- CRITERIO DE REGRESIÓN

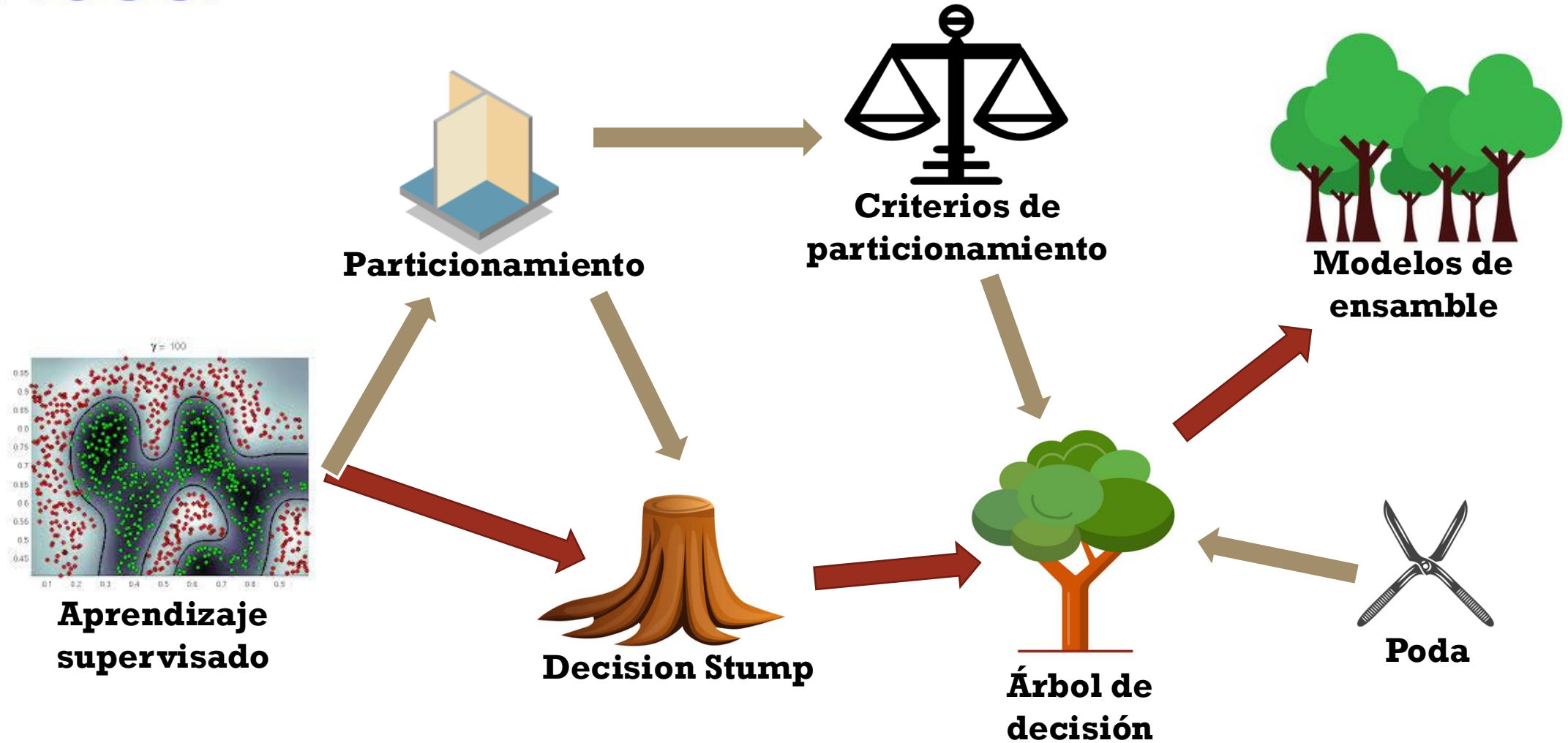
- Para el nodo m , los criterios comunes a minimizar son el MAE, el MSE y la desviación media de Poisson. En el caso del MSE se tendría que

$$H(Q_m) = \frac{1}{n_m} \sum_{y \in Q_m} (y - \bar{y}_m)^2$$



APRENDIZAJE SUPERVISADO



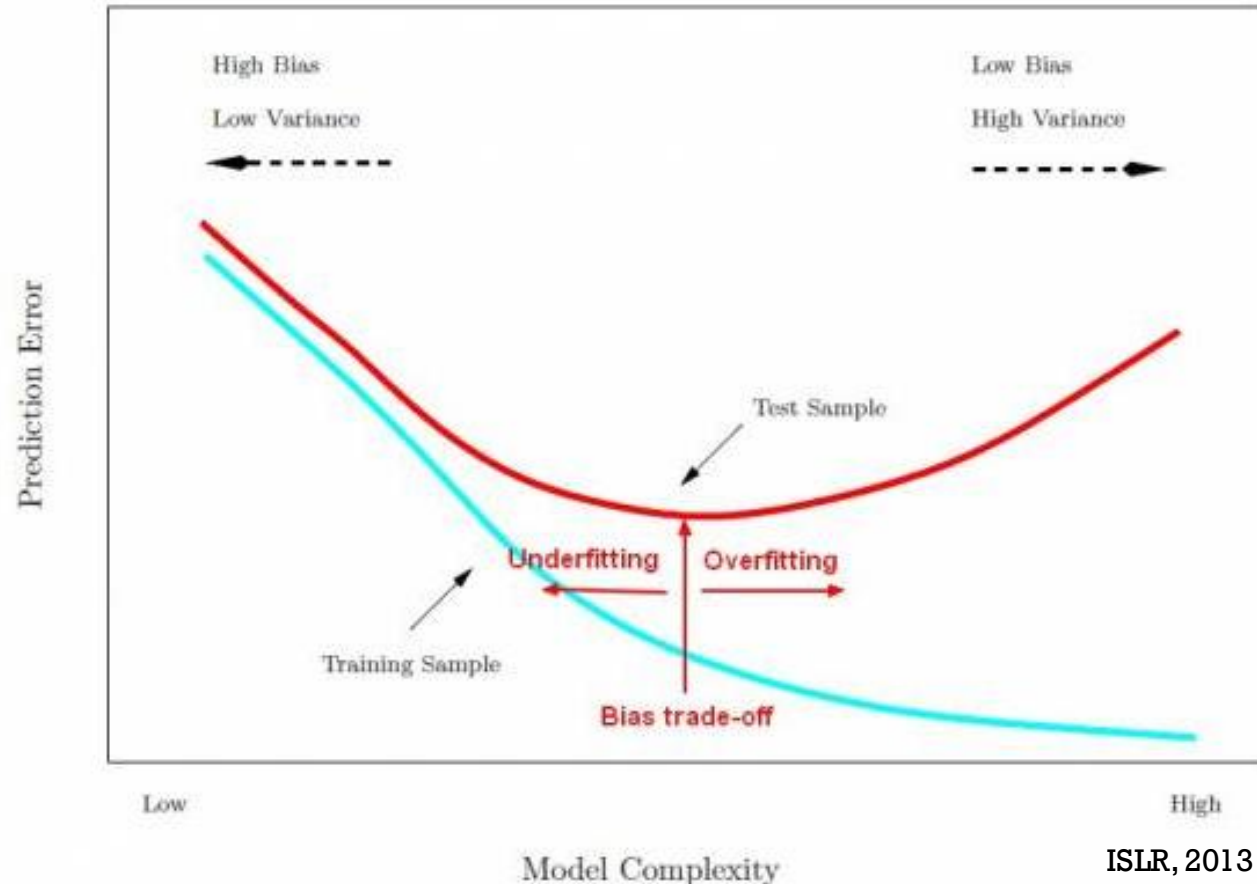
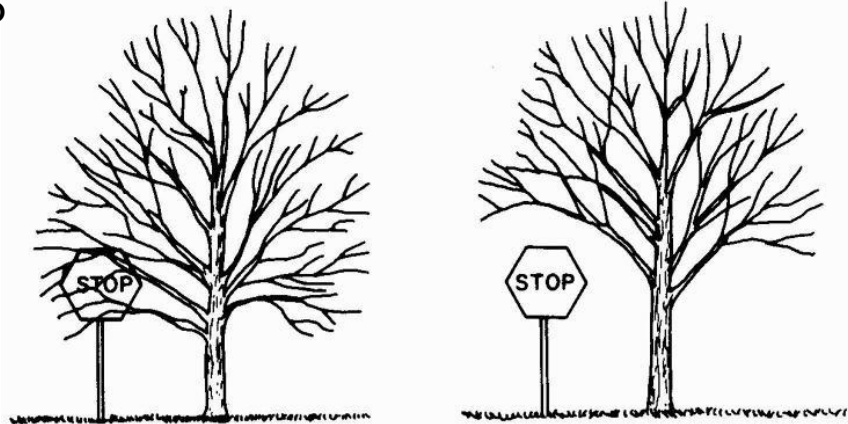


PODA



Pre-Poda: Criterios de parada temprana

- Máxima **profundidad**: limita el crecimiento del árbol de manera global → usa CV
- Mínimo **número de instancias** para permitir particionamiento: limita localmente en cada nodo el crecimiento del árbol → usa CV
- No continuar si no se mejora en el criterio de **particionamiento** o el **error** de entrenamiento → peligroso



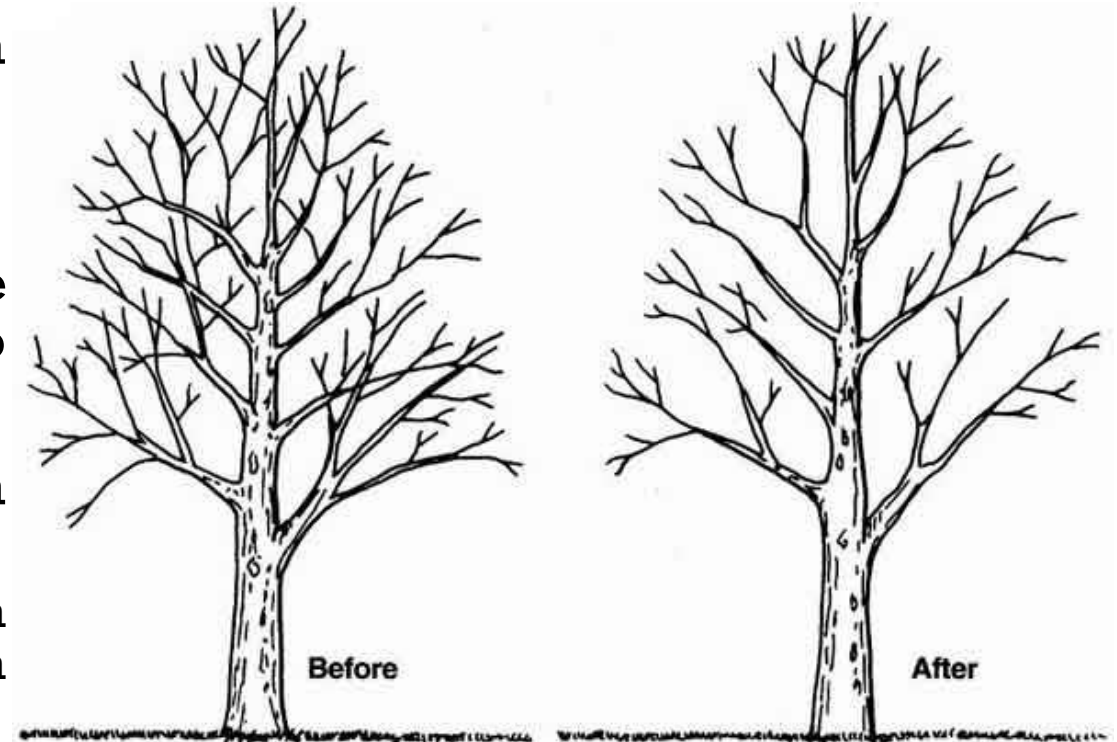
ÁRBOLES DE DECISIÓN: PODA

Post-poda: simplificar el árbol una vez se haya terminado de aprender:

- Preferible a la **pre-poda**
- Complejidad en términos de número de nodos hoja (terminales) $L(T)$, no necesariamente de profundidad
- Podar bottom-up, teniendo en cuenta ahora también la complejidad del árbol:

Costo = Criterio(T) + $\alpha * L(T)$, donde α controla la complejidad del modelo (α se puede estimar a través de CV).

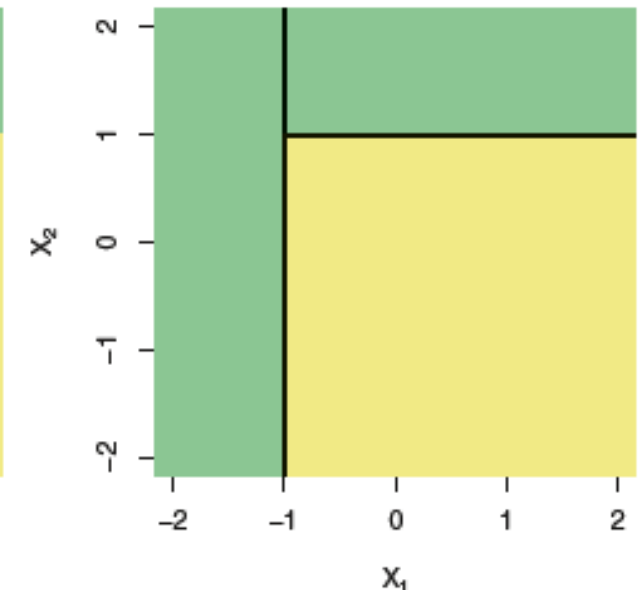
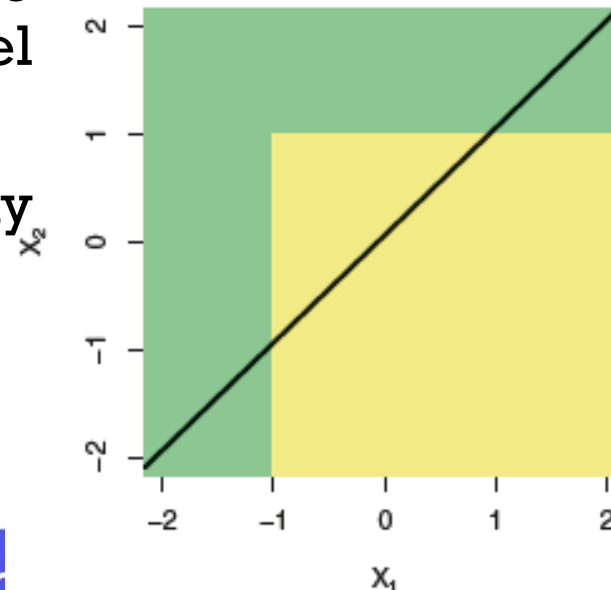
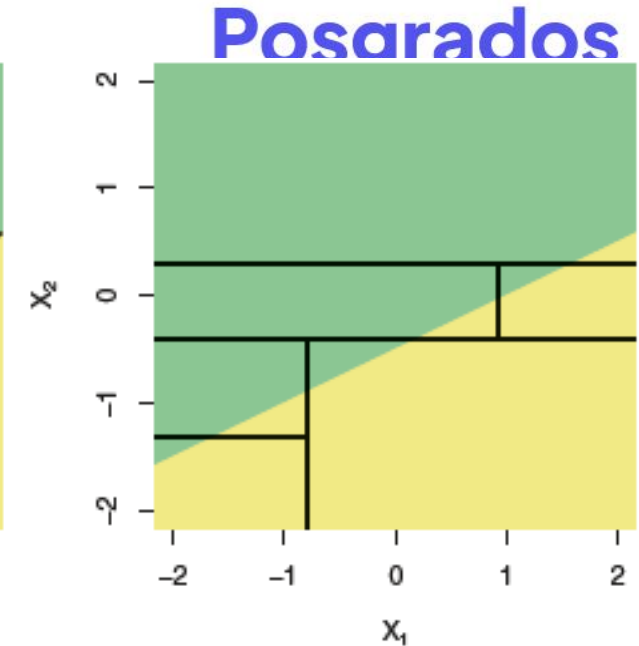
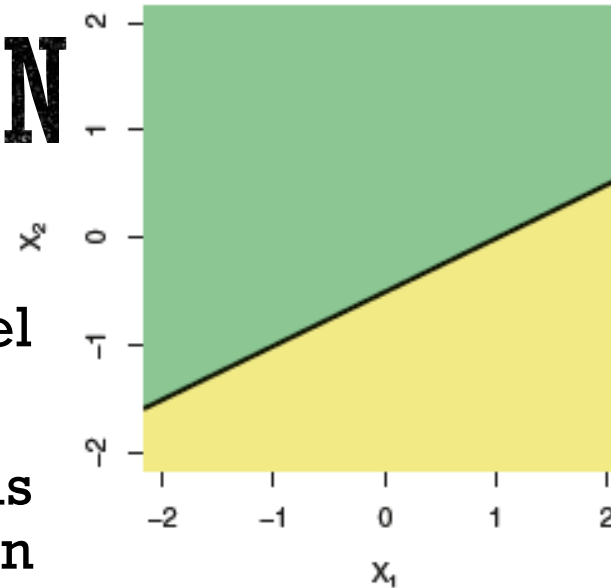
$$\text{Criterio}(T) = \sum_{m=1}^{|T|} \sum_{x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$



ÁRBOLES DE DECISIÓN

Consideraciones finales:

- Su rendimiento depende del comportamiento de los datos
- No son los algoritmos más competitivos, pero producen un modelo simple, altamente interpretable y similar al razonamiento humano
- Tienden al overfitting, por lo que hay que utilizar estrategias de poda



Consideraciones finales:

- Indiferentes a escalamiento, distribución de los datos y a datos faltantes
- Permiten la consideración de atributos cualitativos sin necesidad de crear variables adicionales
- **Crean un ranking de importancia de todas las variables predictivas**
- **Excluyen automáticamente variables no óptimas en el particionamiento**
- Sensibles a bases de datos desbalanceadas y a pequeños cambios en los datos: sesgo hacia los atributos con mayor cantidad de valores
- Muy populares por su simplicidad, representación intuitiva y capacidad de predicción
- Base de **modelos de ensamble** (Bagging: Random Forest), que mejoran el poder predictivo, mientras reducen el error de varianza (overfitting) o el error de sesgo (Boosting)



TALLER: SOBREAPRENDIZAJE DE ARBOLES DE DECISION

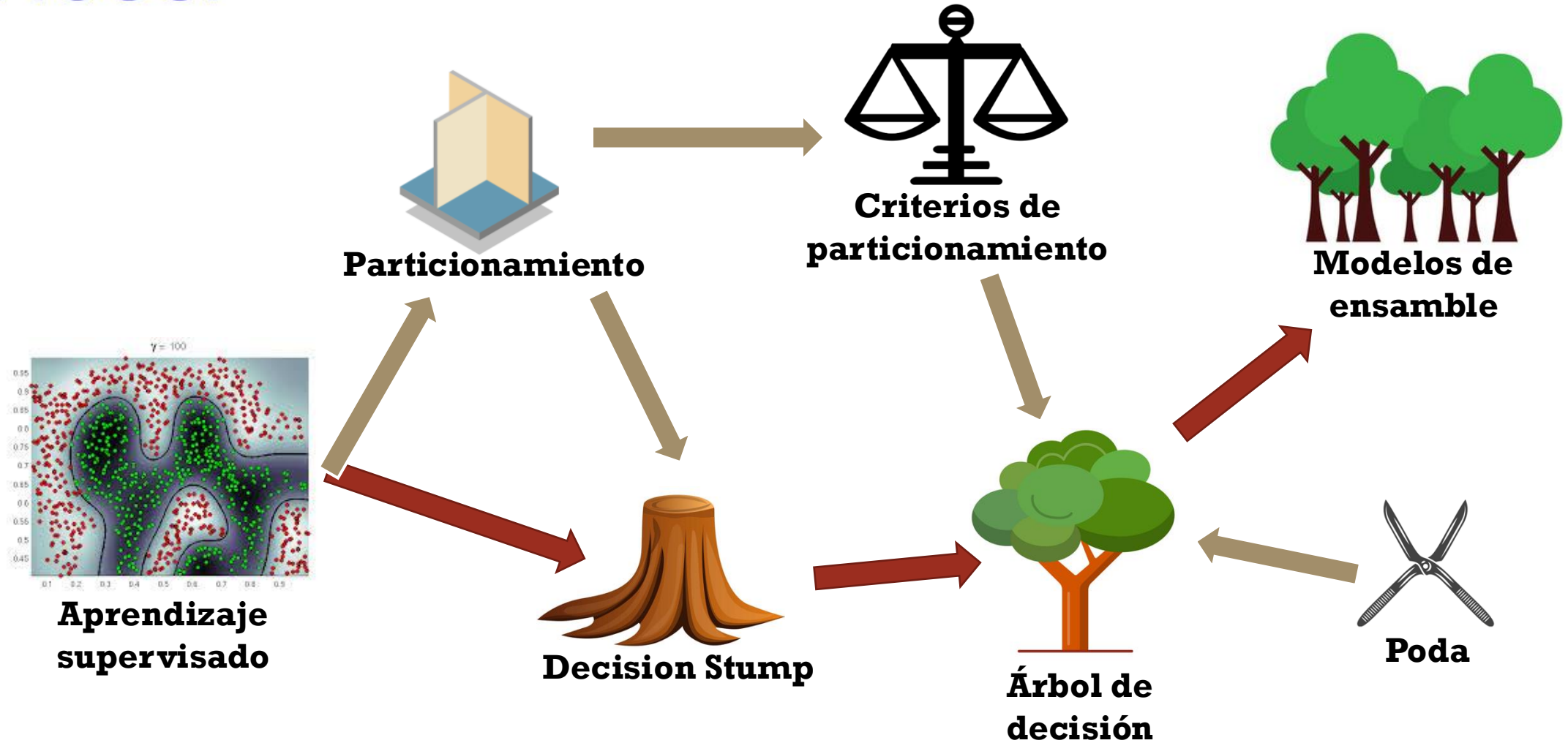
- Revisar el notebook `Interactive_Decision_Tree.ipynb` del profesor Michael Pyrcz.



TALLER: CLASIFICACIÓN CHURN

- **DATASET:** base de datos de 20000 clientes que han abandonado (churn) o no una compañía. La idea es poder predecir en un futuro quiénes son los clientes más propensos a hacer churn, para poder desarrollar campañas que lo prevengan.
- Descargar el dataset 03-churn.csv y trabajar en el cuaderno S2_Arboles_Churn
- Desarrollar el cuaderno hasta llegar a la sección de ensambles





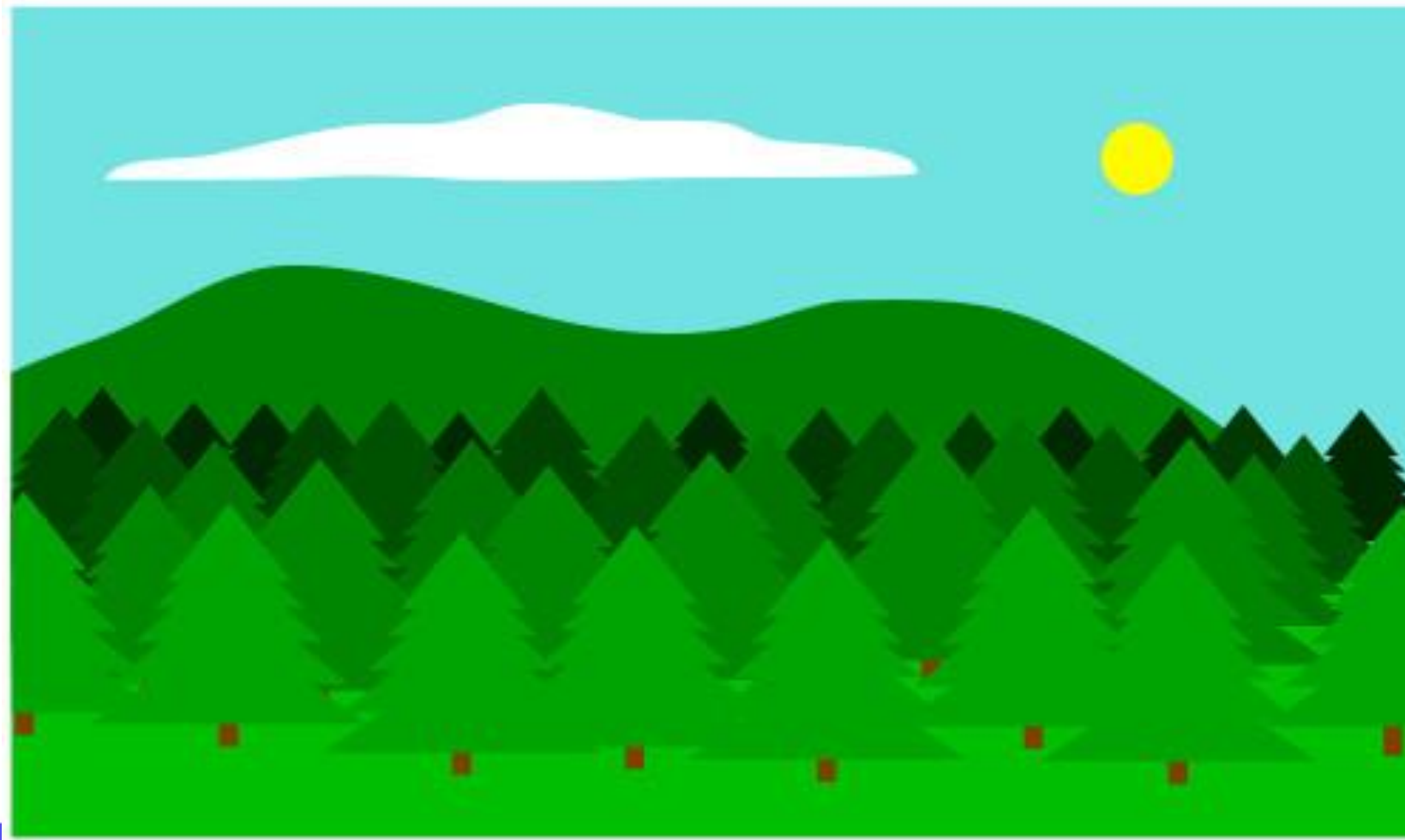


Universidad

icesi

Posgrados

ENSEMBLE LEARNING



Facultad Barberi de Ingeniería, Diseño y Ciencias Aplicadas

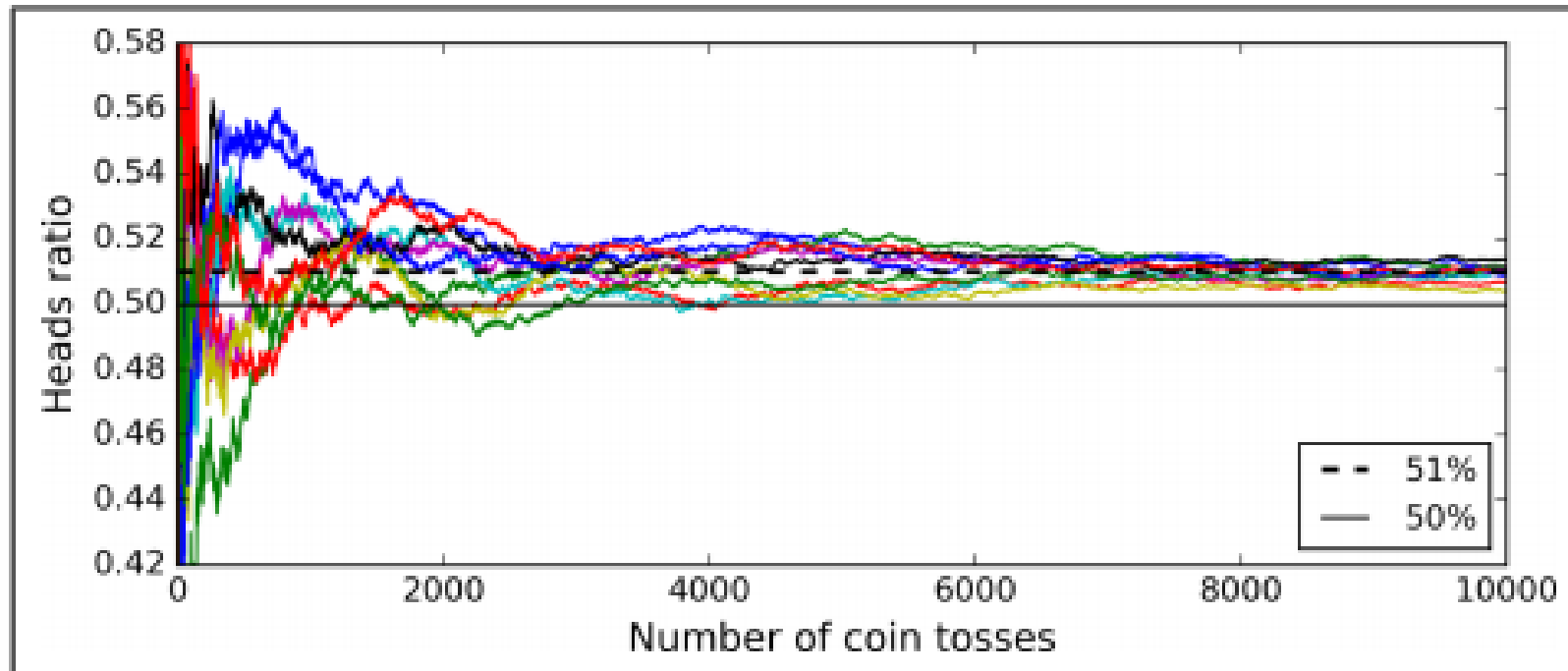


ENSEMBLE LEARNING

¿Cómo verificar que una moneda no esté trucada?

→ Lanzar la moneda n veces

→ A mayor número de lanzamientos, mejor la estimación



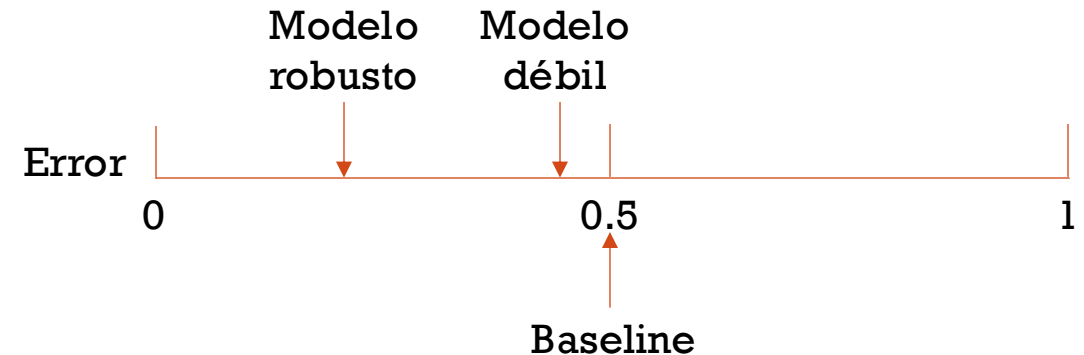
ENSEMBLE LEARNING

- Diversificación:
 - Cada modelo tiene un riesgo de mala predicción
 - ¿Por qué limitar la decisión a un solo modelo si podemos utilizar varios?
 - Combinar un grupo de clasificadores/regresores
 - Decisión basada en la agregación de varios modelos (max promedio)
- Reducir la varianza global al agregar un conjunto de modelos supervisados, y obtener una mayor capacidad de generalización
- Se aumenta la calidad de la predicción, pero se pierde en interpretabilidad

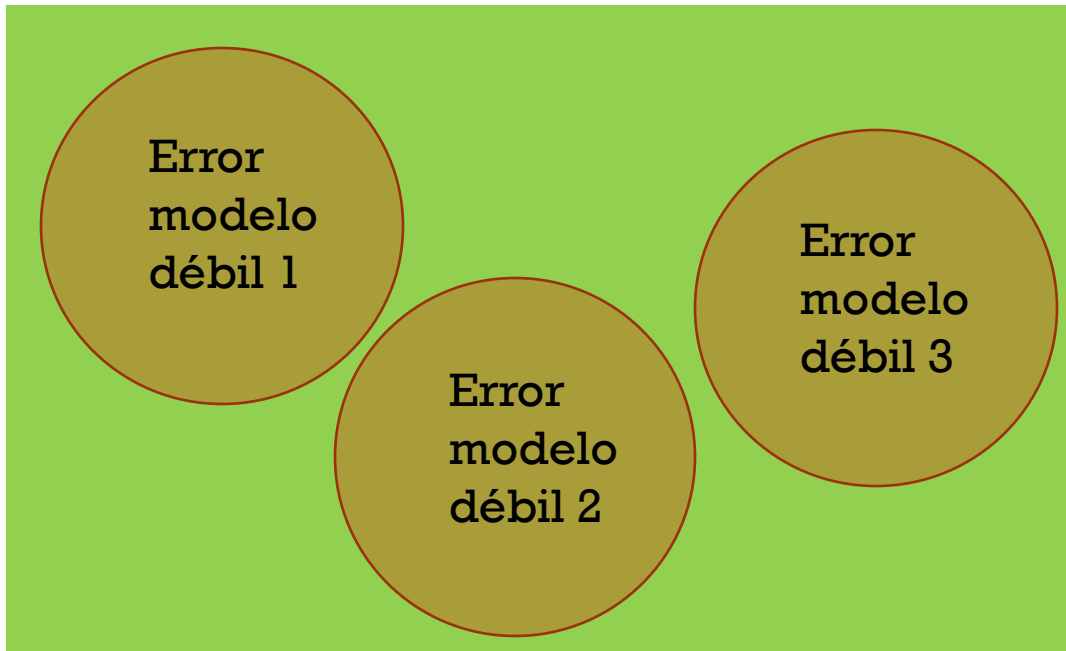


ENSEMBLE LEARNING

- Modelos débiles vs. robustos:
 - Los modelos robustos tenderán a estar de acuerdo en la mayoría de los casos
 - A menudo los datos permiten construir modelos que son apenas mejores al baseline



BAGGING (BOOTSTRAP)



- ¿Cómo es nuestra predicción si ponemos a votar a 3 modelos?
- Entre más de-correlacionados estén los modelos individuales, mejor va a ser el meta-modelo agregado



BAGGING

- Bagging = Bootstrap aggregating
- Bootstrap: técnica de muestreo
 - Consideración de varios conjuntos de entrenamiento/test utilizando **muestreo con reemplazo**
 - Por lo general los muestreos tienen el mismo tamaño del conjunto original
 - Reduce la varianza

Original Dataset

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
-------	-------	-------	-------	-------	-------	-------	-------	-------	----------

Bootstrap 1

x_8	x_6	x_2	x_9	x_5	x_8	x_1	x_4	x_8	x_2
-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

Bootstrap 2

x_{10}	x_1	x_3	x_5	x_1	x_7	x_4	x_2	x_1	x_8
----------	-------	-------	-------	-------	-------	-------	-------	-------	-------

Bootstrap 3

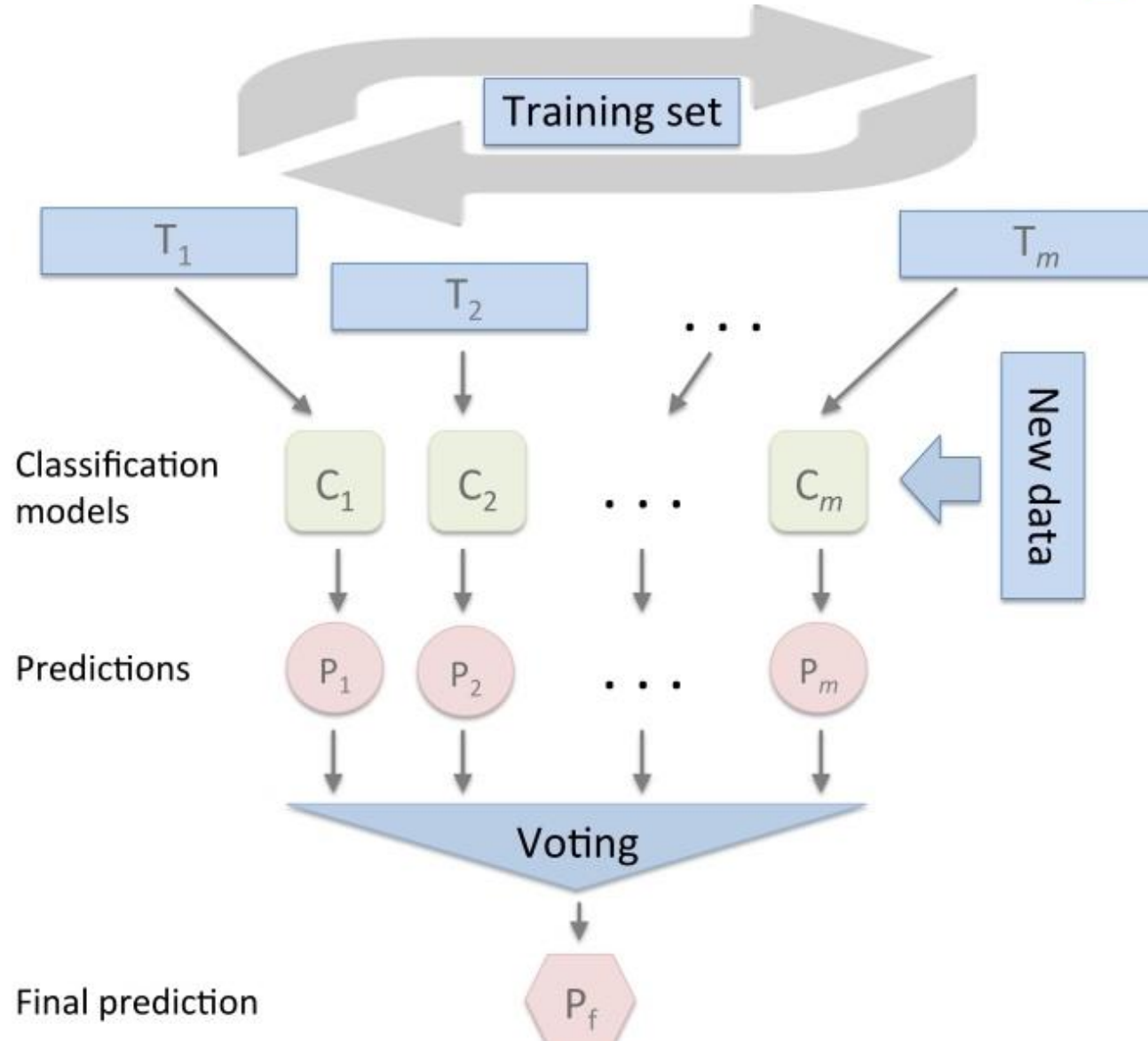
x_6	x_5	x_4	x_1	x_2	x_4	x_2	x_6	x_9	x_2
-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

Sebastian Raschka, 2015



BAGGING

Bootstrap
samples



Raschka, 2017

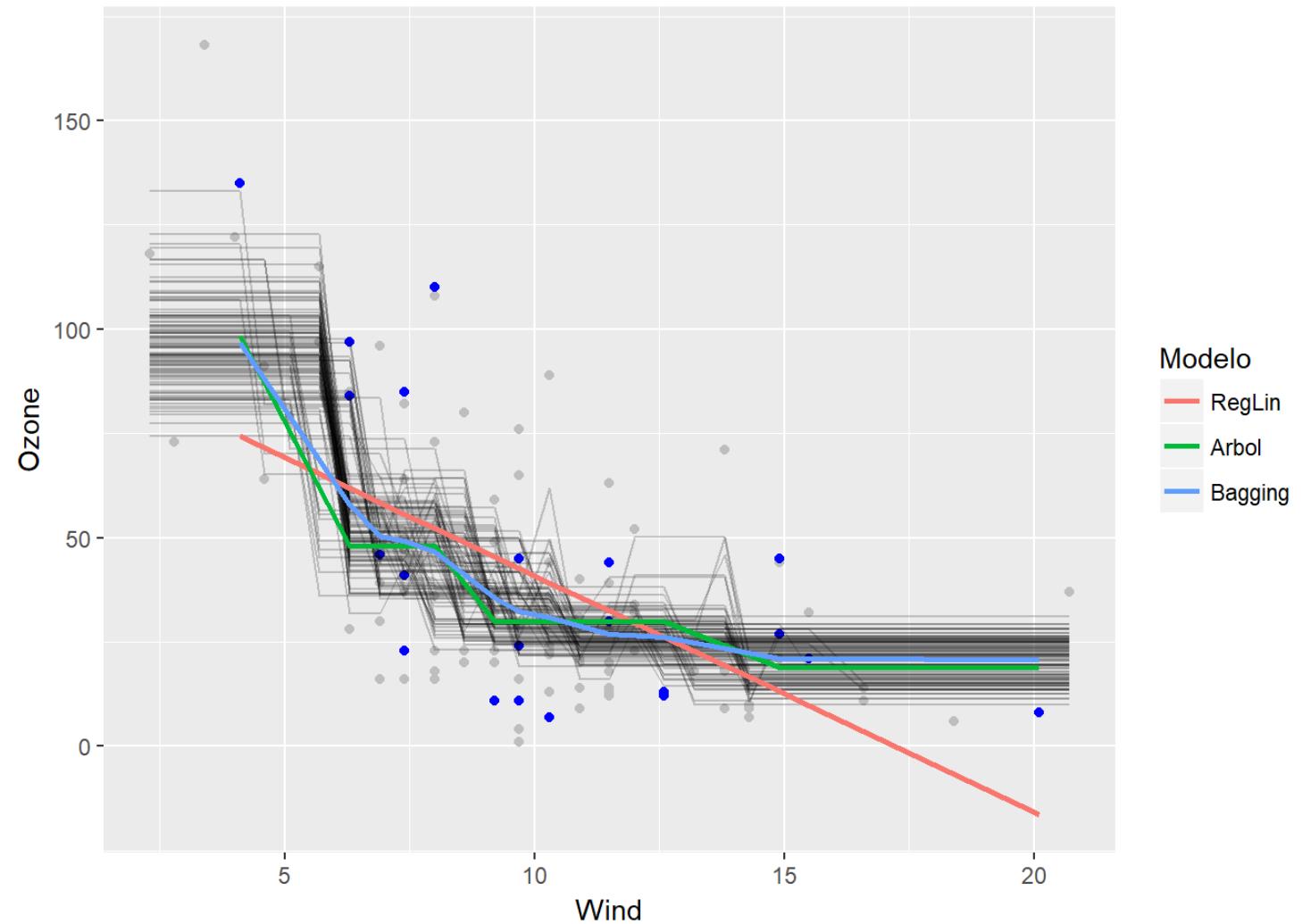
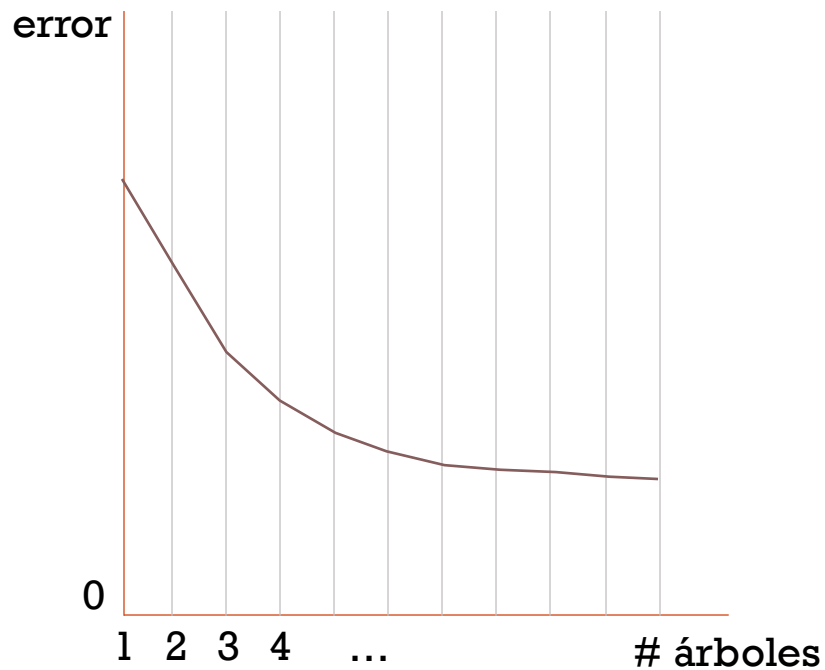


BAGGING

- Algoritmo (Breiman, 1994-1996):
 1. Escoger múltiples subconjuntos de instancias con repetición
 2. Entrenar varias instancias de un mismo tipo de modelo “**robusto**” de aprendizaje (bajo sesgo – alta varianza) basado en cada subconjunto
 3. Agregar las decisiones de cada modelo en una sola decisión global (votación/promedio)
- Muy útil cuando se trata de modelos no lineales
- Resultado: sesgo similar, reducción de varianza (estabilización de las predicciones) → mejora el accuracy
- Se disminuye la interpretabilidad

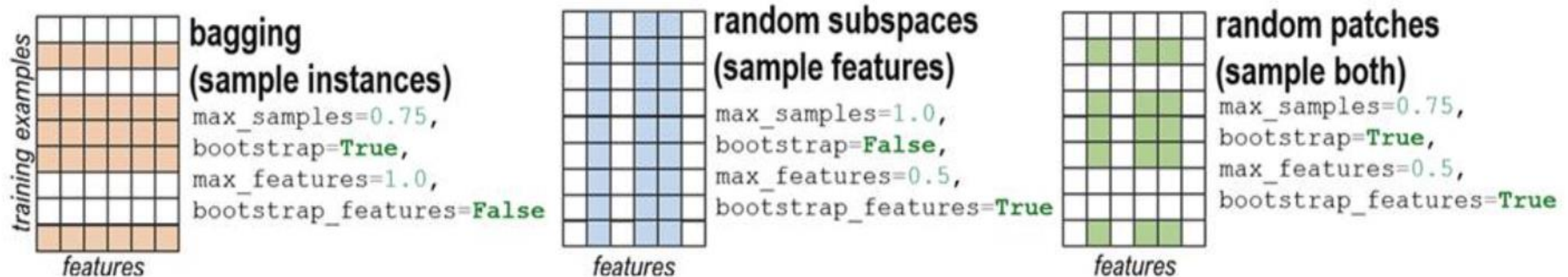


BAGGING



- Se aplica tanto a clasificadores como a regresores

Figure 2.9 Bagging compared to random subspaces and random patches.



- Random patches:** subconjuntos tanto de los registros como de las variables independientes

Random patching



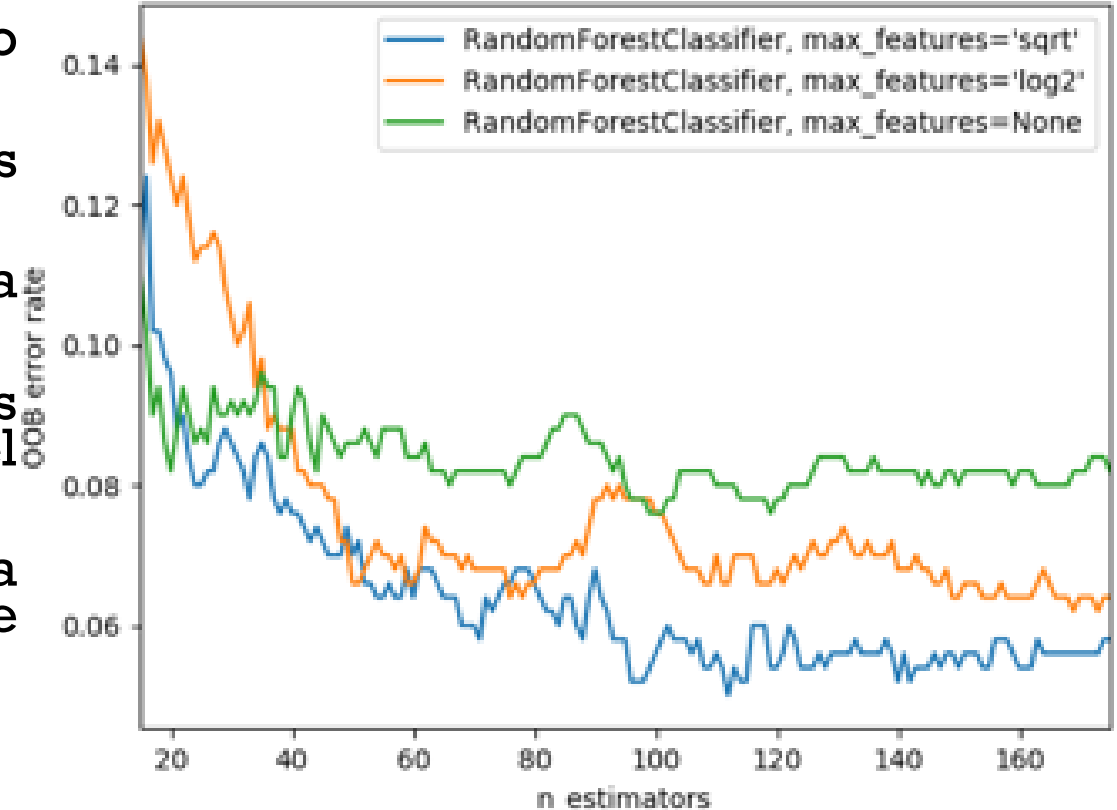
BAGGING

OOB (Out of bag error estimation):

- Un subconjunto bootstrap contiene, en promedio, aproximadamente $2/3$ de los datos del conjunto original
- OOB usa el $1/3$ restante de datos para calcular métricas de evaluación de los modelos individuales, que luego deben agregarse
- Con una cantidad elevada de árboles, OOB tiende hacia el error LOOCV :
 - Evita tener que dejar un test set independiente
 - Evita el costoso proceso de K-Fold CV



- Junto a boosting, tiene un mejor rendimiento en casos de datos **estructurados**
- Extensión de bagging basado en árboles CART
- Cada árbol es **independiente** y tiene la **misma importancia** en la decisión final
- La idea es minimizar la **correlación** entre los modelos, forzando la diferencia más allá del bootstrap de los datos de entrenamiento
- Limita el **número m de atributos** a considerar en cada nodo de particionamiento.
- Valores por defecto:
 - En caso de regresión: $m/3$
 - En caso de clasificación: $\sqrt{m}$

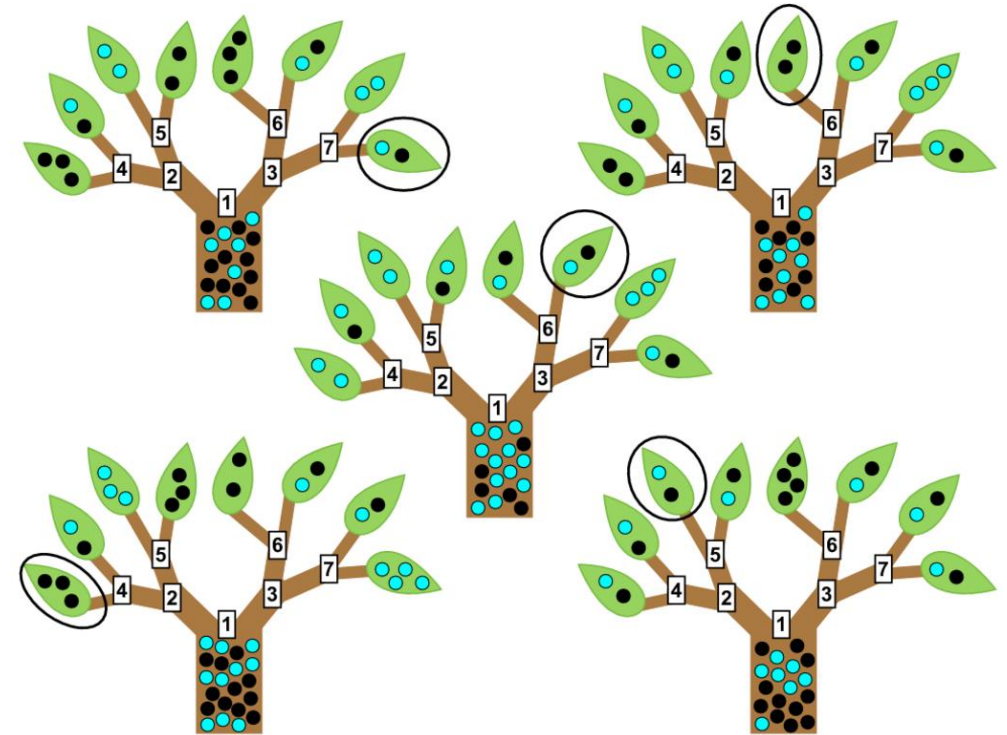


Scikit-learn.org



■ Algoritmo

1. Crear múltiples árboles de decisión, contruidos sobre muestras con reemplazo (bootstrapping) del dataset
2. En cada punto de partición, considerar solo una muestra aleatoria de todas las variables disponibles
3. Crecer el árbol
4. Agregar los resultados de todos los árboles creados



<http://inspirehep.net/record/1335130/plots>

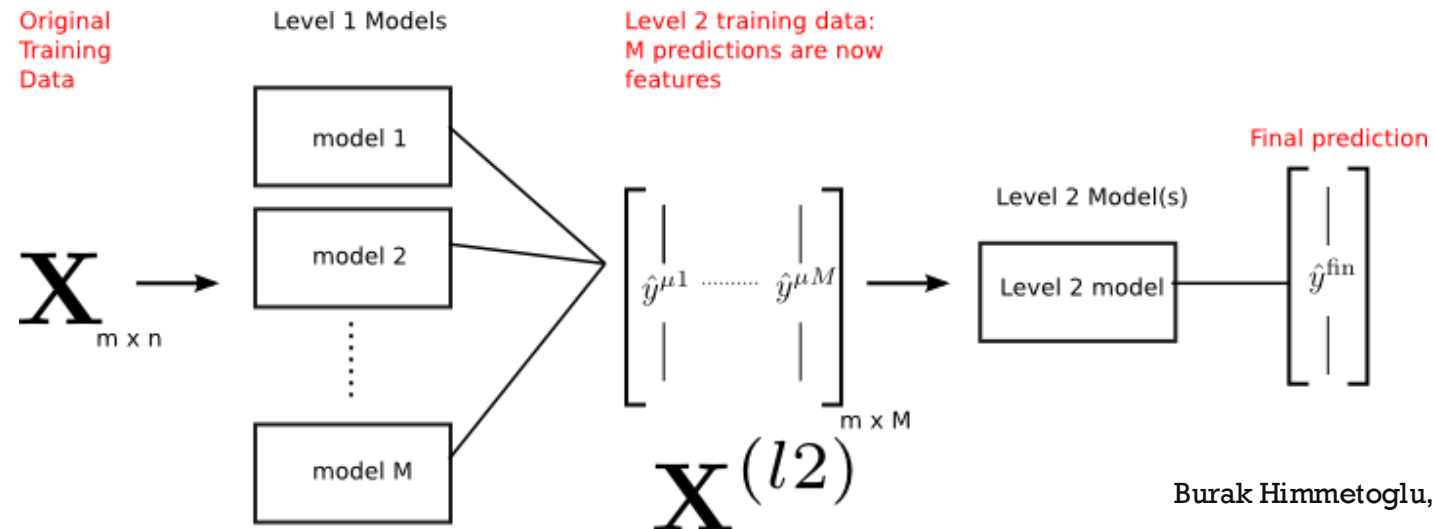


RANDOM FOREST

- Consideraciones
 - Al igual que el bagging:
 - **Heredan** cosas buenas de los árboles de decisión (importancia de las variables), mejoran las malas (varianza)
 - **Lento** (gran número de árboles a entrenar) y toman mucho espacio en memoria, pero es **escalable** (independencia)
 - No genera **overfitting** con más árboles, pero los árboles individuales pueden tenerlo (limitar su profundidad)
 - Muy difíciles de interpretar
 - Requieren de un buen afinamiento de parámetros (bagging es más simple)



- 2 capas de modelos:
 - Una primera capa incluye modelos diferentes, no necesariamente débiles, no necesariamente de la misma familia, que se entrenan con el dataset original
 - Una segunda capa con un solo modelo, típicamente una regresión logística (en clasificación) o lineal (en regresión), que se entrena con las salidas de los modelos de la primera capa como entradas
 - Los modelos de las 2 capas comparten la misma variable objetivo

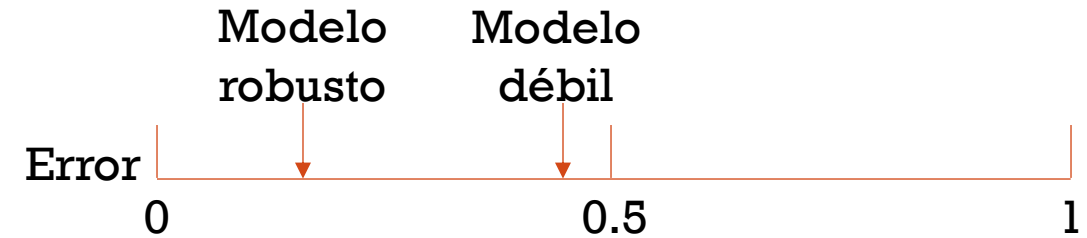
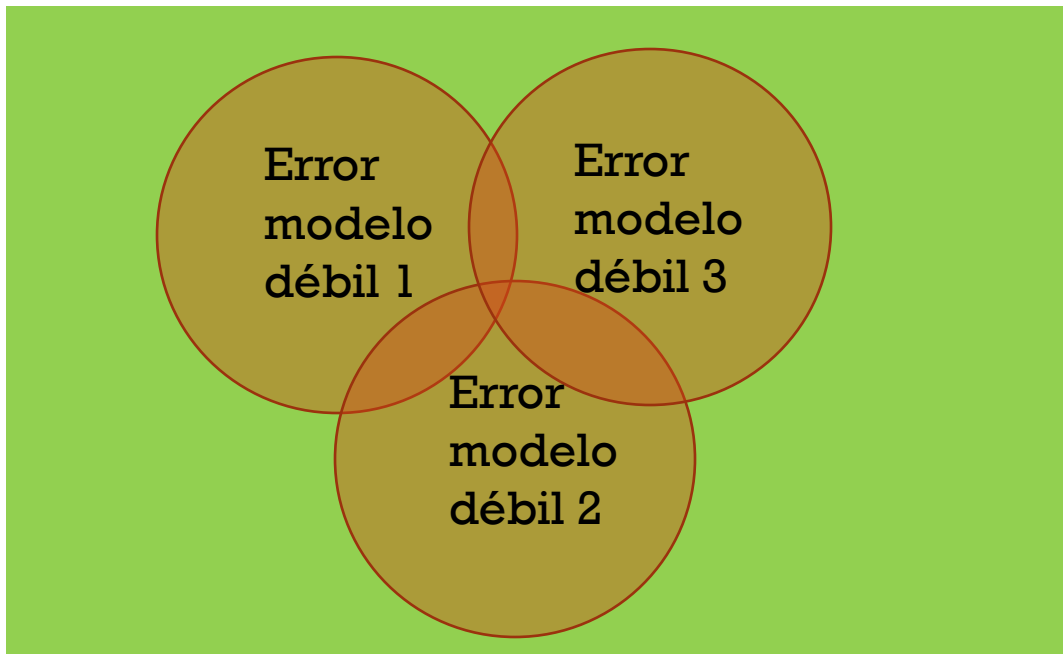


Burak Himmetoglu, UC Santa Barbara.



BOOSTING

- Modelos débiles vs. robustos



- ¿Hay ciertas regiones más importantes en cuanto al error que otras?
- ¿Cómo mejoro en las regiones más susceptibles al error donde ningún modelo logra buenos resultados?



BOOSTING

- Agregación de predictores **débiles**, aprendidos **secuencialmente**
- Aprendizaje lento progresivo: reduce el error de sesgo al considerar los errores previos en los modelos subsiguientes.
- La **importancia de cada registro** se modifica según los resultados del último modelo entrenado → Cada modelo es **dependiente** de los modelos anteriores
- Cada modelo tiene **diferente influencia** en la decisión final; este depende de su poder predictivo
- La idea es reducir el error de sesgo al considerar los errores previos en los modelos subsiguientes
- Puede caer en **overfitting** si el número de árboles (modelos) es grande



■ Algoritmo

1. Crear un modelo predictor **débil** inicial sobre el dataset
2. Asignar un **peso** a la decisión del modelo dado su **error** de entrenamiento
3. Actualización de los pesos de cada instancia del dataset: se aumenta en las instancias que fueron mal predichas, se reduce en las instancias correctas
4. Crear un nuevo predictor, influenciado por los errores cometidos por los modelos anteriores
5. Agregar una decisión ponderada de todos los predictores creados

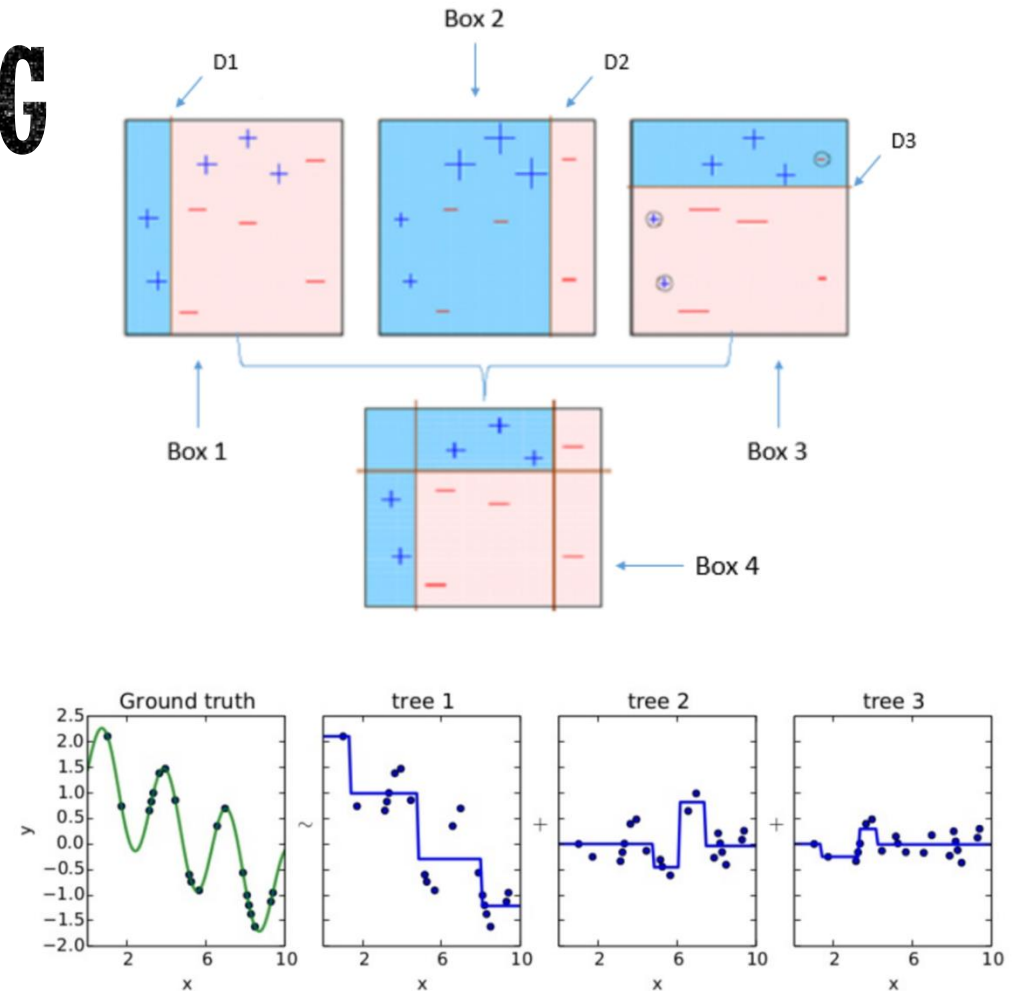
■ Tres parámetros importantes:

- **Número de árboles** a desarrollar
- **Learning rate o shrinking parameter**: controla la velocidad del aprendizaje para limitar el overfitting (valores típicos 0.01 - 0.001)
- **Número de particiones**: Controla la complejidad de los árboles. Se conoce como interacción de profundidad máxima (1 funciona bien)



VARIANTES DE BOOSTING

- ADA BOOST
- GRADIENT BOOSTING
- EXTREME GRADIENT BOOSTING (XGBOOST)
 - Hands-On Machine Learning with Scikit-Learn and TensorFlow. Aurélien Géron. 2017. O'Reilly Medi

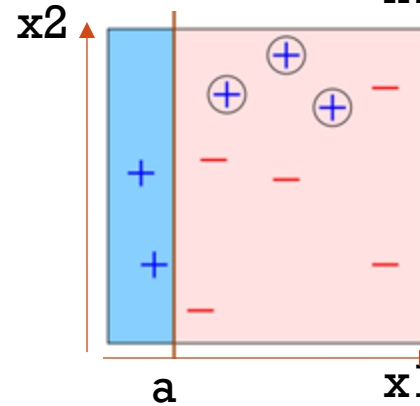
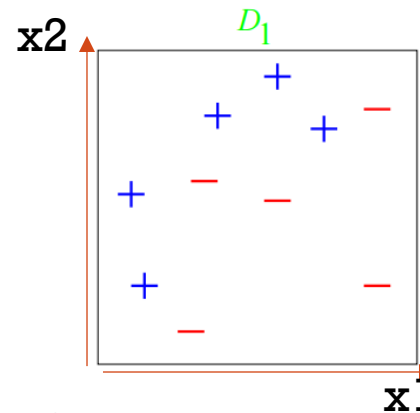


Source: [Quora](#)

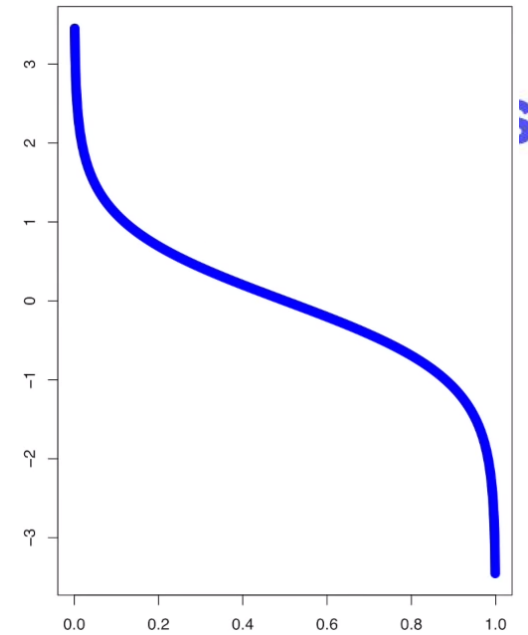


ADA BOOST

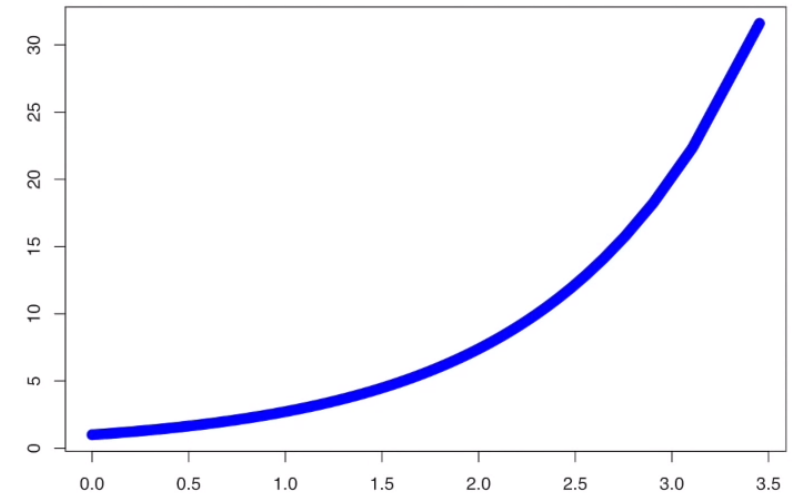
- Combinación de varios **decision stumps**, con **importancias diferentes** en la agregación final
- Inicialmente todas las instancias tienen el mismo peso: $1/n$
- Se encuentra el mejor decisión stump,
- Se evalúan los errores y se calcula la importancia del stump, en el lugar del **learning rate**: $\alpha = \frac{1}{2} \ln \frac{1-\text{error}}{\text{error}}$
- Actualizar los pesos de las instancias
 - Incorrectas: $\text{peso}_t = \text{peso}_{t-1} * e^{\alpha}$
 - Correctas: $\text{peso}_t = \text{peso}_{t-1} * e^{-\alpha}$



Árbol	Error	α
$x_1 < a$	3/10	0.424



α según el error



Nuevo peso dado α



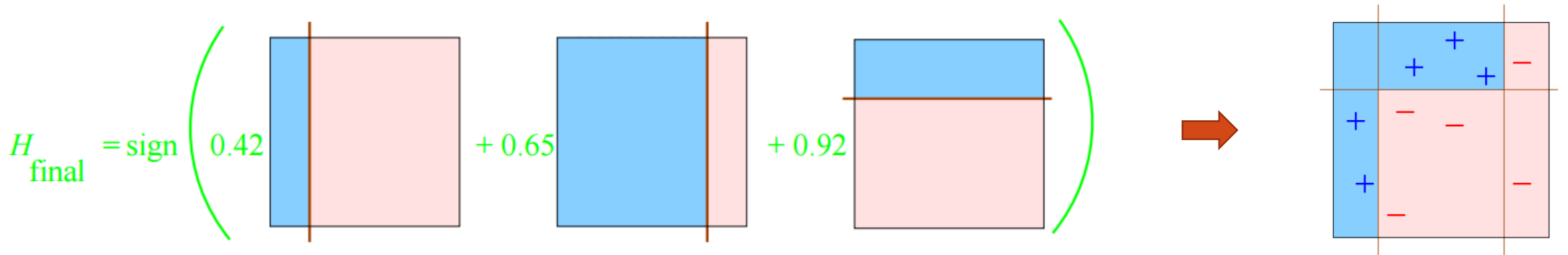
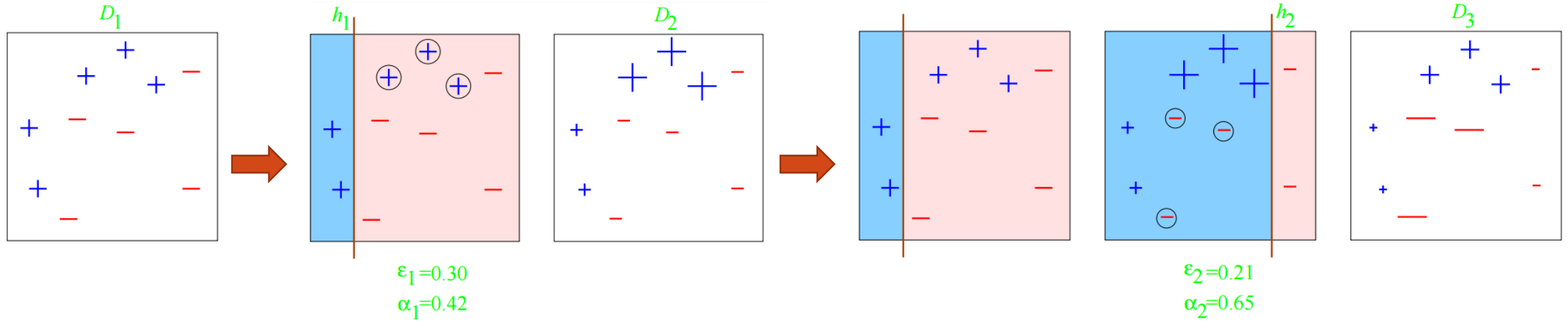


Universidad

ICESI

ADA BOOST

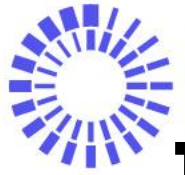
Posgrados



TALLER: BAGGING, RANDOM FOREST, BOOSTING

- Vamos a analizar diferentes configuraciones de bagging y random forest para crear un clasificador del conjunto de datos 03-churn.csv, considerando diferentes valores de los hiper-parámetros #árboles y depth
- Continuar con el notebook S2_Arboles_Churn desde la sección de ensambles





Universidad

icesi

REFERENCIAS

Posgrados

- *An Introduction to Statistical Learning with Applications in Python. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani. Springer. 2023.*
- *Müller, A. C., & Guido, S. (2016). Introduction to machine learning with Python: a guide for data scientists. "O'Reilly Media, Inc".*
- *Diapositivas de clase*
- *Guías de lectura*
- *Cuadernos Jupyter (Intu)*

